Sistema de identificación y localización espacial de objetos basado en visión por computador para asistencia a personas con discapacidad visual

Diomedes Farid Cerda Leon

Universidad del Magdalena

Facultad de Ingeniería

Programa de Ingeniería de Sistemas

Santa Marta, Colombia

2026

Sistema de identificación y localización espacial de objetos basado en visión por computador para asistencia a personas con discapacidad visual

Diomedes Farid Cerda Leon

Trabajo presentado como requisito parcial para optar al título de:

Ingeniero de Sistemas

Director (a):

Idanis Beatriz Diaz Bolaño

Codirector (a):

Aquiles Alfonso Cohen Llanes

Línea de Investigación:

Inteligencia artificial y visión por computador aplicadas a tecnologías de asistencia

Grupo de Investigación:

Desarrollo y gestión de tecnologías para las organizaciones y la sociedad – TecnOS

Universidad del Magdalena

Facultad de Ingeniería

Programa de Ingeniería de Sistemas

Santa Marta, Colombia

2026

AGRADECIMIENTOS

A mi abuela, por su cariño y por el lugar tan importante que ocupa en mi vida. A mi padre, por su apoyo y por acompañarme en este camino. A mis amigos, por su compañía, sus palabras de ánimo y los momentos compartidos durante esta etapa. A toda mi familia, por estar presente y ser parte de este logro. Mi agradecimiento principal es para ustedes: este trabajo también representa lo que significa contar con las personas que quiero. Agradezco, además, a Idanis Beatriz Diaz Bolaño, directora, y a Aquiles Alfonso Cohen Llanes, codirector, por la orientación académica del proyecto, y a la Universidad del Magdalena por el espacio de formación.

Resumen

Se diseñó e implementó AssistiveVision, un sistema móvil Android orientado al apoyo de la percepción de entornos interiores para personas con discapacidad visual. Integra captura manual, preprocesamiento, detección de diez clases de objetos, selección configurable, localización relativa en seis zonas y descripción auditiva local en español e inglés. Su arquitectura modular incorpora CameraX, EfficientDet-Lite2 y síntesis de voz con sherpa-onnx; procesa y anuncia la escena bajo demanda, sin conexión a internet. El desarrollo comprendió revisión conceptual, selección del detector, integración y evaluación técnica. Se compararon seis configuraciones sobre 80 imágenes y 257 anotaciones; EfficientDet-Lite2 obtuvo media de precisión promedio con intersección sobre unión mínima de 0,50 (mAP50) de 0,648 y latencia aislada de 124,9 ms en la unidad central de procesamiento (CPU, central processing unit) de Redmi Note 13. Se verificó la lógica con 34 pruebas de la máquina virtual de Java (JVM, Java Virtual Machine) y se reprodujeron 198 fotografías mediante instrumentación Android en emulador Pixel 8. Las 889 anotaciones y 33 grupos de captura (G01–G33, en 30 recintos físicos) permitieron comparar montajes y lentes: palo de selfie 0,5x alcanzó 82 detecciones correctas y pecho 1x, una puntuación F1, media armónica de precisión y exhaustividad, de 61,9 %. También se evaluaron tiempos de procesamiento, brillo, oclusión, distancia estimada y la retroalimentación auditiva (alcanzando 90,7 % de precisión en la lista de hasta dos objetos anunciados y superando cinco pruebas de síntesis local en modo avión). El resultado es un prototipo integrado, verificado y reproducible.

Palabras clave: detección de objetos; tecnología de asistencia; Android; EfficientDet; cobertura de objetos; voz local.

ABSTRACT

AssistiveVision was designed and implemented as an Android mobile system to support perception of indoor environments for people with visual impairments. It integrates manual capture, preprocessing, detection of ten object classes, configurable selection, relative localization in six image regions, and offline spoken descriptions in Spanish and English. Its modular architecture combines CameraX, EfficientDet-Lite2, and speech synthesis through sherpa-onnx to process and announce scenes on demand without internet connectivity. Development covered conceptual review, detector selection, integration, and technical evaluation. Six configurations were compared using 80 images and 257 annotations; EfficientDet-Lite2 achieved mean average precision at an intersection-over-union threshold of 0.50 (mAP50) of 0.648 and isolated central processing unit (CPU) latency of 124.9 ms on a Redmi Note 13. System logic was checked with 34 Java Virtual Machine (JVM) tests, and Android instrumentation replayed 198 photographs on a Pixel 8 emulator. The 889 annotations and 33 capture groups (G01–G33, across 30 physical rooms) supported mount and lens comparisons: the selfie stick with tripod 0.5x yielded 82 correct detections, while chest harness 1x achieved an F1 score, the harmonic mean of precision and recall, of 61.9%. Processing times, digital brightness, occlusion, estimated distance, and auditory feedback (achieving 90.7% precision in the two-object spoken list and passing five offline speech instrumentation tests in airplane mode) were also evaluated. The outcome is an integrated, verified, and reproducible prototype.

Keywords: object detection; assistive technology; Android; EfficientDet; object coverage; offline speech.

Contenido

Pág.

Resumen IV
Abstract V
Contenido VI
Lista de figuras VIII
Lista de tablas IX
Lista de símbolos X
Introducción 1
1. Marco de referencia: detección de objetos para la asistencia visual 5
1.1 Detección de objetos y métricas de evaluación 5
1.2 Trabajos relacionados: sistemas de asistencia visual 8
1.3 Arquitecturas de detección: one-stage y two-stage 10
1.4 Modelos ligeros y eficiencia para dispositivos móviles 11
1.5 Cuantización de modelos: Float32 y Float16 12
2. Selección experimental del modelo de detección 15
2.1 Metodología 15
2.2 Criterios de selección para la evaluación comparativa 16
2.3 Modelos candidatos 16
2.4 Entorno experimental 17
2.5 Metodología de evaluación 18
2.6 Resultados 19
2.7 Análisis de resultados 22
2.8 Selección del modelo final 24
3. Diseño e implementación del prototipo AssistiveVision 25
3.1 Arquitectura general del sistema 25
3.2 Posicionamiento del dispositivo 26
3.3 Módulo de captura 26
3.4 Módulo de preprocesamiento 27
3.5 Módulo de detección 27
3.6 Módulo de localización espacial 28
3.7 Módulo de filtrado y priorización 29
3.8 Módulo de audio y descriptores de voz 30
3.9 Modos de operación 31
3.10 Configuración del sistema 31

3.11 Interfaz de usuario **31**
3.12 Verificación y rendimiento del prototipo **34**
3.13 Requerimientos y criterios de verificación **35**
4. Evaluación técnica del sistema y de las condiciones de captura **37**
4.1 Diseño y protocolo de evaluación **37**
4.2 Comparación de posiciones de montaje **43**
4.3 Comparación de montaje y lente **46**
4.4 Detección por bandas de distancia estimada **48**
4.5 Desempeño por clase y selección para la salida de voz **50**
4.6 Consideraciones de validez **53**
4.7 Distribución del tiempo de procesamiento **55**
4.8 Brillo digital y oclusión observada **55**
4.9 Evaluación de la retroalimentación auditiva **58**
5. Conclusiones **61**
Anexo A. Repositorio y reproducción de los experimentos **65**
Referencias **67**

Lista de figuras

Pág.

Figura 3.1. Flujo de procesamiento de la implementación actual..... 25
Figura 3.2. Tipos de soporte utilizados para posicionar el dispositivo..... 26
Figura 3.3. Regiones de la imagen para la localización relativa..... 29
Figura 3.4. Secuencia de interacción manual y liberación de la captura..... 32
Figura 3.5. Pantallas de captura y configuración de AssistiveVision..... 33
Figura 4.1. Ejemplos de espacios y condiciones de captura..... 39
Figura 4.2. Ilustración de la colocación de los soportes..... 41
Figura 4.3. Objetos correctos por fotografía y montaje..... 45
Figura 4.4. Objetos correctos por fotografía en las seis configuraciones..... 47
Figura 4.5. Recall por distancia estimada e intervalo bootstrap del 95 %..... 49
Figura 4.6. Precisión, recall y F1 del detector y de la lista de hasta dos objetos..... 52

Lista de tablas

Pág.

Tabla 1.1. Notación y unidades de lectura.....8
Tabla 2.1. Resultados de evaluación en equipo de escritorio.....20
Tabla 2.2. Desglose de AP50 por clase para cada modelo.....20
Tabla 2.3. Resultados de benchmark en Android utilizando CPU.....21
Tabla 2.4. Resultados de benchmark en Android utilizando GPU Delegate.....21
Tabla 2.5. Comparación de candidatos finales.....24
Tabla 3.1. Clases de objetos prioritarias anunciadas por el sistema.....28
Tabla 3.2. Zonas de localización espacial y descriptores asociados.....29
Tabla 3.3. Parámetros configurables del sistema.....31
Tabla 3.4. Requerimientos operativos y evidencia de verificación.....35
Tabla 4.1. Tipologías observables y ejemplos de grupos.....38
Tabla 4.2. Orden y tamaño de las configuraciones de captura.....40
Tabla 4.3. Diseño y unidades de análisis.....43
Tabla 4.4. Resultados agrupados por montaje.....43
Tabla 4.5. Pruebas t pareadas por montaje.....44
Tabla 4.6. Resultados por configuración.....46
Tabla 4.7. Comparación de conteos con el palo de selfie 0,5x.....47
Tabla 4.8. Recall por banda de distancia estimada.....49
Tabla 4.9. Resultados por clase en el conjunto completo.....51
Tabla 4.10. Distribución de tiempos de reproducción en emulador en milisegundos.....55
Tabla 4.11. Detección agrupada por brillo digital de las fotografías.....56
Tabla 4.12. Recall por nivel de oclusión con etiquetas asistidas por IA.....57

LISTA DE SÍMBOLOS

AP	Precisión promedio sobre la curva precisión-recall
COCO	Common Objects in Context
EXIF	Metadatos de captura de la fotografía
F1	Media armónica de precisión y recall
FN	Falso negativo
FP	Falso positivo
FPS	Fotogramas por segundo
GT	Anotación de referencia o verdad de campo
IC	Intervalo de confianza
IoU	Intersección sobre unión de cajas
m	Metro
ms	Milisegundo
px	Píxel
RGB	Canales rojo, verde y azul
TP	Verdadero positivo

Introducción

La discapacidad visual constituye una limitación significativa para la autonomía de quienes la padecen, en particular cuando se trata de desplazarse e interactuar en espacios interiores, donde la densidad de obstáculos, el mobiliario y la variabilidad de las escenas dificultan la anticipación del entorno. De acuerdo con la Organización Mundial de la Salud, al menos 2,2 mil millones de personas presentan deficiencia visual o ceguera, y de ellas al menos 1.000 millones tienen una condición que podría haberse prevenido o aún no ha sido atendida (World Health Organization, 2019). Frente a esta situación, los sistemas de asistencia basados en visión por computador se han consolidado como una alternativa tecnológica viable para apoyar la percepción del entorno, al convertir la información visual capturada por una cámara en descripciones auditivas, como exploran los prototipos descritos en (Guravaiah et al., 2023; Meenakshi et al., 2022; More et al., 2025).

En los últimos años, la disponibilidad de modelos de detección de objetos optimizados para dispositivos móviles, junto con frameworks de inferencia local como TensorFlow Lite (TFLite) (TensorFlow, s. f.), ha permitido ejecutar tareas de visión por computador directamente sobre teléfonos inteligentes de gama media, sin depender de conectividad a internet ni de servidores externos. Este avance abre la posibilidad de construir ayudas técnicas de bajo costo, portátiles y funcionales para personas con discapacidad visual, especialmente en contextos donde el acceso a soluciones comerciales especializadas es limitado.

En este contexto, el presente trabajo de grado aborda el diseño, la implementación y la evaluación de un sistema de asistencia visual ejecutado sobre un dispositivo móvil Android, capaz de identificar objetos prioritarios de un entorno interior, determinar su ubicación espacial relativa y anunciar esta información al usuario mediante descriptores de voz.

La detección de objetos mediante redes neuronales profundas ha evolucionado desde arquitecturas de dos etapas, como Faster Region-based Convolutional Neural Network (Faster R-CNN) (Ren et al., 2015), hasta detectores de una etapa orientados al tiempo real, como You Only Look Once (YOLO) (Redmon et al., 2016), Single Shot MultiBox Detector (SSD) (Liu et al., 2016) y EfficientDet (Tan et al., 2020), que ofrecen diferentes compromisos entre exactitud y costo computacional. En el ámbito de las tecnologías de asistencia, trabajos recientes han demostrado la viabilidad de ejecutar detectores de objetos sobre Android con fines de apoyo a personas con discapacidad visual; es el caso del sistema propuesto por More et al. (2025), que integra un modelo YOLOv8 convertido a TensorFlow Lite con retroalimentación auditiva. Estos antecedentes evidencian tanto la pertinencia del problema como la necesidad de comparar experimentalmente los modelos bajo condiciones de despliegue concretas. La revisión incluye además aplicaciones que combinan soportes corporales, estimación aproximada de distancia y voz (Meenakshi et al., 2022), y sistemas que conectan reconocimiento y síntesis de etiquetas (Guravaiah et al., 2023). Estos antecedentes orientan la comparación hacia la detección, su ejecución móvil y la comunicación de la información.

El trabajo busca desarrollar un sistema de asistencia visual en un dispositivo móvil que identifique objetos de entornos interiores y comunique su ubicación relativa mediante descriptores de audio, como apoyo a la percepción del entorno de personas con discapacidad visual. Este propósito requiere seleccionar un modelo de detección optimizado para su ejecución móvil e integrarlo con la cámara y la síntesis de voz en un prototipo funcional para Android. La evaluación del prototipo en entornos interiores controlados se orienta a analizar la exactitud de detección, la latencia de procesamiento y la claridad de la retroalimentación auditiva, para valorar la identificación de los objetos y la información que se busca transmitir al usuario.

El sistema se circunscribe a entornos interiores y a un conjunto de diez clases de objetos prioritarios presentes de manera frecuente en espacios domésticos e institucionales: persona, silla, sofá, cama, mesa, inodoro, televisor, computador portátil, lavamanos y refrigerador. La retroalimentación al usuario se realiza exclusivamente por vía auditiva, en español e inglés, mediante la síntesis de voz disponible en el propio dispositivo. El prototipo opera en un modo manual, en el que el usuario solicita la descripción de la escena en el momento que lo requiere. El desarrollo no contempla el entrenamiento de modelos desde

cero, sino la selección, evaluación e integración de modelos preentrenados de acceso libre.

El desempeño depende del dispositivo, el encuadre, la iluminación y las oclusiones. La localización describe regiones bidimensionales del campo visual de la cámara, entendido como la parte de la escena registrada en la imagen, y no estima distancias ni garantiza movilidad segura. El capítulo 4 evalúa las configuraciones de captura reproducidas en Android, los tiempos de procesamiento, la distancia aproximada y la retroalimentación auditiva; las pruebas de navegación en campo con usuarios finales corresponden al trabajo futuro.

El resultado del proyecto es un prototipo funcional que demuestra la viabilidad de ejecutar un flujo integrado de captura, detección, selección de objetos, localización relativa y descripción auditiva de escenas interiores sobre un teléfono inteligente de gama media, sentando las bases para una ayuda técnica de apoyo a la percepción del entorno para personas con discapacidad visual. En el contexto institucional, este propósito se relaciona con los compromisos de la Universidad del Magdalena con la innovación tecnológica, la investigación y la inclusión, expresados en el Plan de Desarrollo UNIMAGDALENA Comprometida 2020-2030 (Universidad del Magdalena, 2020) y en el Plan de Gobierno Rectoral 2024-2028 (Universidad del Magdalena, 2024). El prototipo constituye, además, una base para trabajos futuros de evaluación con usuarios finales.

El capítulo 1 presenta la literatura, los conceptos, las métricas y las arquitecturas de detección. El capítulo 2 presenta la metodología y documenta la selección inicial del modelo y sus mediciones en escritorio y Redmi Note 13. El capítulo 3 describe la implementación actual del prototipo. El capítulo 4 analiza las 198 fotografías por montaje, lente y banda de distancia estimada, junto con el tiempo de procesamiento y la retroalimentación auditiva. El capítulo 5 reúne conclusiones, recomendaciones y trabajo futuro; un único anexo permite localizar el código y reproducir la evaluación; las instrucciones de uso se mantienen en los README.

1. Marco de referencia: detección de objetos para la asistencia visual

Este capítulo presenta el marco de referencia para el desarrollo de un sistema de asistencia visual basado en detección de objetos. Primero se presentan los conceptos y las métricas de evaluación y se revisan trabajos relacionados con aplicaciones móviles y dispositivos de apoyo para personas con discapacidad visual; después se describen las arquitecturas de detección de una y dos etapas, las estrategias de eficiencia para dispositivos móviles y el paradigma DETR. La revisión permite relacionar las decisiones del prototipo AssistiveVision con los problemas y resultados reportados en la literatura.

1.1 Detección de objetos y métricas de evaluación

Una red neuronal profunda (DNN, deep neural network) aprende representaciones para asignar clases y localizar objetos en una imagen. La detección produce cajas, clases y puntuaciones; esas puntuaciones ordenan la confianza del modelo y no se interpretan como probabilidades calibradas de acierto. La evaluación compara predicciones con una referencia anotada mediante las reglas siguientes.

Cada objeto anotado se representa mediante una caja delimitadora y una clase. La intersección sobre unión (IoU, intersection over union) compara una caja predicha con una de referencia: IoU = área de intersección / área de unión. Su valor varía de 0 a 1. Una coincidencia requiere la clase correcta y un solapamiento que alcance el umbral elegido; exigir IoU ≥ 0,50 no significa simplemente cubrir la mitad del objeto. Las anotaciones y el evaluador siguen el formato de Microsoft Common Objects in Context (COCO) (COCO dataset contributors, s. f.; Lin et al., 2014).

La precisión promedio (AP, average precision) resume la curva de precisión frente a exhaustividad para una clase al variar el umbral de puntuación. La media de precisión

promedio entre clases (mAP, mean average precision) agrega las categorías evaluables. En este informe, mAP50 exige IoU ≥ 0,50; mAP50-95 promedia además diez umbrales, de 0,50 a 0,95 con incrementos de 0,05. El segundo indicador es más exigente respecto al ajuste geométrico. Las tablas por clase conservan AP50; los resultados agregados se denominan uniformemente mAP50 y mAP50-95, aunque los archivos del evaluador COCO utilicen las claves AP50 y AP50-95 (COCO dataset contributors, s. f.; Lin et al., 2014).

Una detección emparejada correctamente es un verdadero positivo (TP, true positive); una predicción sin correspondencia es un falso positivo (FP, false positive), y una instancia anotada sin detección correcta es un falso negativo (FN, false negative). El emparejamiento es uno a uno: una anotación solo puede justificar un TP y una predicción duplicada puede convertirse en FP. Un error de clase no recupera la instancia de la clase verdadera (COCO dataset contributors, s. f.). La precisión responde a qué proporción de las predicciones es correcta: P = TP / (TP + FP) (Manning et al., 2008). Por ejemplo, si se anotan cinco objetos y el detector produce cuatro predicciones, tres correctas y una incorrecta, TP = 3, FP = 1 y FN = 2. La precisión es 75 % y el recall es 60 %. Este ejemplo es ilustrativo y no pertenece a los resultados experimentales.

Recall, o exhaustividad, es la proporción de instancias anotadas visibles que el detector recupera: TP/(TP+FN) (Manning et al., 2008). Los falsos negativos describen objetos presentes en la imagen y no detectados; un objeto fuera del encuadre no entra en el denominador. Por ello, este trabajo acompaña recall con los conteos de objetos visibles y detecciones correctas. Estas métricas no miden directamente riesgo de colisión ni seguridad de navegación.

La puntuación F1 corresponde a la media armónica entre precisión y recall: F1 = 2 × (Precision × Recall) / (Precision + Recall). A diferencia de la media aritmética, penaliza el desequilibrio entre ambas métricas, por lo que resulta útil como métrica única de exactitud global cuando se requiere un compromiso entre falsos positivos y falsos negativos (Manning et al., 2008).

Los fotogramas por segundo (FPS, frames per second) expresan cuántas imágenes procesa el modelo en un segundo, determinando la frecuencia con la que el sistema puede actualizar su percepción del entorno. La latencia de inferencia corresponde al tiempo en milisegundos requerido para procesar una imagen completa, desde que entra al modelo

hasta que se obtienen las detecciones. Para inferencias secuenciales aisladas, los FPS se calculan como 1000 dividido por la latencia en milisegundos; esta conversión no incluye captura, interfaz ni voz (TensorFlow contributors, s. f.).

La memoria de acceso aleatorio (RAM, random access memory) se reporta como la huella aproximada medida por la Benchmark Tool durante la inicialización y ejecución, en megabytes (MB) (TensorFlow contributors, s. f.). Incluye efectos del proceso de medición y no representa exclusivamente el tamaño del modelo o de sus tensores. Se utiliza para comparar las configuraciones bajo el mismo procedimiento.

Tiempo de inicialización. El tiempo de inicialización corresponde al intervalo requerido para cargar el modelo en memoria, crear el intérprete y preparar los delegados de hardware antes de la primera inferencia. Esta métrica afecta directamente el tiempo que transcurre desde que el usuario abre la aplicación hasta que el sistema de detección está listo para operar (TensorFlow contributors, s. f.).

Tabla 1.1. Notación y unidades de lectura.

Símbolo	Significado y convención
B = [x, y, w, h]	Caja anotada en píxeles: esquina superior izquierda, ancho y alto.
W, H; cx, cy	Ancho y alto del fotograma original; coordenadas del centro de la caja.
u, v	Centro normalizado: u = cx/W; v = cy/H. Origen arriba a la izquierda.
g (G01–G33); c	Grupo de captura (1 a 33, abreviado de G01 a G33); configuración de montaje y lente (1 a 6).
GT	Referencia anotada (ground truth); en tablas de conteos, número de instancias anotadas.
TP, FP, FN	Detecciones correctas, predicciones sin correspondencia e instancias no recuperadas.
P, R, F1	Precisión, exhaustividad y su media armónica. Se expresan como razón o porcentaje.
Δg; n	Diferencia entre dos configuraciones del mismo grupo; tamaño de muestra del análisis.
α; t; gl; p	Nivel de significación, estadístico t, grados de libertad y valor p.
IC 95 %	Intervalo de confianza del 95 %. En el capítulo 4 procede del bootstrap de grupos.
d; f_px; H_cls; h_px	Distancia aproximada, focal en píxeles, altura de referencia de clase y alto de caja.
m, ms, px, MB	Metro, milisegundo, píxel y megabyte. 0,30 equivale a 30 %.

Nota. El punto decimal de los archivos equivale a la coma decimal del texto. Los intervalos de confianza y los grados de libertad se explican al presentar el método estadístico.

1.2 Trabajos relacionados: sistemas de asistencia visual

Los trabajos revisados muestran que una solución de asistencia visual debe resolver simultáneamente tres problemas: detectar objetos relevantes, comunicar su presencia de manera comprensible y operar con recursos limitados. La literatura también evidencia que la estimación de distancia y la selección de la información que se anuncia son retos distintos de la clasificación de objetos (Guravaiah et al., 2023; Meenakshi et al., 2022; More et al., 2025).

Meenakshi et al. (2022) desarrollaron una aplicación móvil para Android orientada a apoyar la movilidad de personas con discapacidad visual mediante YOLOv3. El sistema integra identificación de objetos, estimación aproximada de la distancia, determinación de la dirección y generación de instrucciones de voz. El trabajo propone el uso del teléfono en un soporte corporal, como un cinturón de pecho, y demuestra la pertinencia de integrar detección y retroalimentación en un dispositivo de uso cotidiano. Sin embargo, sus cálculos de distancia dependen del tamaño aparente del objeto y de una calibración aproximada; además, los autores reconocen limitaciones para el mapeo completo de la ruta.

Guravaiah et al. (2023) presentaron Third Eye, un sistema que utiliza YOLOv5 para reconocer objetos y convertir las etiquetas detectadas en voz mediante las bibliotecas pyttsx3 y Google Text-to-Speech (gTTS). El conjunto combina COCO con quince clases adicionales de interés para el entorno cotidiano, como puertas, ventanas, casilleros y señales, para un total de noventa y cinco clases. El aporte principal es la integración explícita entre detección y síntesis de voz; no obstante, la implementación se plantea en Python y no resuelve de manera métrica la profundidad ni la localización espacial detallada.

En un preprint, Pawar (2025) describe una aplicación Android basada en la interfaz de programación de aplicaciones (API, application programming interface) TensorFlow Object Detection y SSD MobileNet, ajustada mediante transferencia de aprendizaje para reconocer ochenta categorías. El sistema utiliza más de mil imágenes anotadas, aumento de datos y retroalimentación mediante texto a voz. El artículo reporta una exactitud aproximada del noventa por ciento y destaca la operación sin conexión como una ventaja de accesibilidad. Para el presente trabajo, este antecedente propone el uso de SSD MobileNet en aplicaciones móviles, pero también resalta la necesidad de separar la exactitud reportada en un conjunto propio de la medición reproducible de mAP, precisión, recall, latencia y FPS.

Figura et al. (2025) compararon Faster R-CNN ResNet-101 v1, YOLOv8s, SSD MobileNetV2 y EfficientDet-Lite2 en un entorno simulado de gafas de realidad aumentada. Evaluaron velocidad, precisión, consumo de recursos y eficiencia energética. Sus resultados muestran que los detectores de una etapa son más apropiados para ejecución directa en el dispositivo; YOLOv8s obtuvo el mejor compromiso global entre las métricas

consideradas, mientras EfficientDet-Lite2 y SSD MobileNetV2 presentaron ventajas de rapidez. Este estudio es especialmente cercano al problema de selección abordado en el capítulo 2 porque compara dos de los candidatos considerados en este trabajo.

More et al. (2025) propusieron una aplicación Android para personas con discapacidad visual que convierte YOLOv8 a TensorFlow Lite y añade retroalimentación auditiva. En sus pruebas, el modelo YOLOv8n a 640×640 alcanzó un mAP50 de 79,9 % y un mAP50-95 de 57,1 %, con un tiempo medio de inferencia de 350 ms; el modelo mediano obtuvo valores superiores de exactitud, pero con un costo temporal mayor. El resultado ilustra el compromiso entre tamaño del modelo, resolución, exactitud y latencia que debe considerarse antes de desplegar un detector en un teléfono.

Estos antecedentes justifican una arquitectura que ejecute la inferencia localmente, limite la información anunciada y traduzca la caja detectada a una zona espacial comprensible. AssistiveVision adopta un modo manual bajo demanda y una localización bidimensional por zonas. El capítulo 4 explora la relación entre detección y distancia estimada; el estudio no incluyó validación con distancias medidas.

1.3 Arquitecturas de detección: one-stage y two-stage

Entre los detectores convolucionales revisados se distinguen enfoques de dos etapas, como Faster R-CNN (Ren et al., 2015), y de una etapa, como YOLO (Redmon et al., 2016) y SSD (Liu et al., 2016).

Faster R-CNN divide el proceso en dos pasos. En la primera etapa, una Red de Propuesta de Regiones (RPN, region proposal network) escanea la imagen y genera regiones candidatas donde podrían existir objetos. En la segunda etapa, se extraen características de cada región candidata mediante redes convolucionales, asignando una etiqueta de clase y ajustando las coordenadas del bounding box (Ren et al., 2015). Esta descripción corresponde al detector de propuestas de regiones utilizado como antecedente.

Los detectores de una etapa procesan la imagen completa en una sola pasada por la red neuronal, prediciendo clases y bounding boxes simultáneamente como un problema de regresión directa (Liu et al., 2016; Redmon et al., 2016). Una red convolucional extrae mapas de características espaciales y semánticas, sobre los cuales el modelo predice

coordenadas y probabilidades de clase. YOLO prioriza la velocidad (Redmon et al., 2016), SSD utiliza mapas multiescala y cajas predefinidas (Liu et al., 2016), y EfficientDet incorpora una fusión bidireccional de características y un escalamiento compuesto (Tan et al., 2020).

El transformador de detección (DETR, DEtection TRansformer) introduce un paradigma basado en transformers para realizar detección de objetos de extremo a extremo (Carion et al., 2020). El modelo procesa la imagen completa y utiliza object queries para producir predicciones directas, sin seguir explícitamente el esquema clásico de una o dos etapas. Su formulación reduce componentes como las cajas predefinidas y la supresión no máxima, en este proyecto se mantuvo como antecedente y no se midió su desempeño en el teléfono objetivo.

1.4 Modelos ligeros y eficiencia para dispositivos móviles

El despliegue en un teléfono introduce restricciones de memoria, energía, almacenamiento y tiempo de respuesta. Por esta razón, la eficiencia no depende únicamente del detector elegido: también intervienen el backbone, la resolución de entrada, la representación numérica y las técnicas de compresión (Howard et al., 2017; Sandler et al., 2018).

Howard et al. (2017) propusieron MobileNets como una familia de redes para visión móvil basada en convoluciones separables en profundidad. La factorización reduce de manera importante las operaciones y los parámetros frente a una convolución convencional. Además, los multiplicadores de ancho y resolución permiten ajustar el costo computacional a las capacidades del dispositivo, lo que explica la amplia adopción de MobileNet como backbone de detectores SSD móviles.

MobileNetV2, presentada por Sandler et al. (2018), introduce residuos invertidos y cuellos de botella lineales para mejorar la relación entre expresividad y costo. Al combinarse con SSDLite, la arquitectura reporta una reducción sustancial de operaciones y parámetros frente a YOLOv2, manteniendo un desempeño competitivo en detección sobre COCO. Este antecedente explica por qué las variantes SSD MobileNet son candidatos naturales para aplicaciones Android.

Dai (2020) aborda la conversión de modelos Float32 a uint8 con TensorFlow Lite para reducir almacenamiento y coste de inferencia en dispositivos de borde. El artículo también reproduce una comparación entre SSD y Faster R-CNN; sus cifras no se utilizan aquí como mediciones del prototipo ni como evidencia aislada del efecto de la cuantización. Este antecedente motiva evaluar velocidad y exactitud en el dispositivo y con el protocolo propios.

Magdy et al. (2025) analizaron la compresión de Faster R-CNN mediante entrenamiento con precisión mixta de coma flotante de 16 bits (FP16), poda no estructurada y cuantización dinámica. En conjuntos de imágenes de teledetección, la estrategia redujo en promedio el tamaño del modelo un 25,6 % y el número de parámetros un 56,6 %, manteniendo el mAP. Aunque el dominio es diferente, el resultado muestra que un detector de dos etapas puede optimizarse; aun así, la complejidad de ese proceso debe contrastarse con la disponibilidad de modelos ligeros preentrenados para el caso de uso móvil.

Estos trabajos sustentan la elección de métricas conjuntas de exactitud y costo computacional. En una aplicación de asistencia, un modelo rápido pero con muchos falsos positivos puede producir anuncios incorrectos, mientras un modelo exacto pero lento no entrega información oportuna. Como criterio de diseño, y sin medir aquí confianza del usuario, la selección experimental del capítulo 2 no se basa en una única métrica.

1.5 Cuantización de modelos: Float32 y Float16

La reducción de precisión numérica, por ejemplo de pesos en coma flotante de 32 bits (Float32) a 16 bits (Float16), puede disminuir el tamaño del archivo. Su efecto sobre la latencia y la exactitud depende del modelo, del backend y del hardware; requiere medición. La cuantización entera, como la representación de enteros de ocho bits (INT8), introduce además escalas y desplazamientos que deben respetarse durante la inferencia. El formato de los pesos y el tipo del tensor de entrada son propiedades distintas (TensorFlow, s. f.).

1.5.1 TensorFlow Lite y GPU Delegate

TensorFlow Lite (TFLite) es el framework de inferencia de Google optimizado para dispositivos móviles y embebidos. Ejecuta modelos en formato .tflite directamente en el dispositivo, sin requerir conexión a internet ni servidores externos (TensorFlow, s. f.).

El delegado de la unidad de procesamiento gráfico (GPU, graphics processing unit) permite delegar operaciones compatibles de TFLite a la GPU, con backends como Open Computing Language (OpenCL) (Google, s. f.-e; Khronos Group, 2013). Las operaciones no admitidas pueden permanecer en la unidad central de procesamiento (CPU, central processing unit) y la división del grafo puede introducir transferencias y sincronización. La documentación advierte de ese coste (Google, s. f.-e); estas posibilidades no identifican por sí solas la causa del rendimiento observado en el dispositivo.

En síntesis, la literatura orienta la comparación hacia modelos preentrenados que equilibren exactitud, velocidad, memoria y facilidad de integración. El capítulo 2 formaliza estos criterios y presenta las mediciones realizadas en escritorio y Android para seleccionar el modelo base del prototipo.

2. Selección experimental del modelo de detección

Este capítulo presenta la selección experimental de una arquitectura de detección optimizada para dispositivos móviles. Primero se presenta la metodología general y se establecen los criterios de selección; después se describen los candidatos, el entorno y el protocolo de evaluación; finalmente se analizan los resultados de escritorio y Android para identificar el modelo más adecuado para el sistema de asistencia visual propuesto.

2.1 Metodología

La metodología conserva las tres fases de la propuesta de pasantía (Cerda Leon, s. f.). La fase 1 comprende revisión de literatura, análisis del problema, requerimientos, selección del modelo, delimitación de objetos y diseño de descriptores. La comparación inicial de seis configuraciones sobre 80 imágenes y 257 anotaciones pertenece a esta fase y se presenta en el capítulo 2. La fase 2 corresponde al diseño integrado y al desarrollo del prototipo Android, descritos en el capítulo 3. La fase 3 evalúa el posicionamiento y el desempeño con 198 fotografías organizadas en 33 grupos de captura (G01–G33); sus resultados se presentan en el capítulo 4. El benchmark inicial en Redmi Note 13 y la reproducción posterior en emulador Pixel 8 tienen alcances distintos y se analizan por separado.

La evaluación de la fase 3 abarca el desempeño de detección, el tiempo de procesamiento y la claridad de la retroalimentación auditiva (evaluada mediante la precisión de la lista seleccionada para voz, la estructura sintáctica y espacial de los descriptores y la síntesis neuronal fuera de línea). De forma complementaria, se analizan la distancia geométrica estimada, el brillo digital y la oclusión observada.

2.2 Criterios de selección para la evaluación comparativa

Antes de comparar los modelos se establecieron tres criterios de selección. El primero es la utilidad para el reconocimiento de objetos, entendida como la capacidad de identificar y localizar varias clases en una imagen mediante *bounding boxes*. El segundo es la compatibilidad con dispositivos móviles, que exige ejecutar la inferencia directamente en un teléfono Android y alcanzar tiempos de respuesta adecuados para el uso previsto. El tercero es la disponibilidad de implementaciones y pesos preentrenados de acceso libre, condición necesaria para integrar y reproducir el prototipo en un contexto académico.

Estos criterios delimitan el conjunto de candidatos y se complementan con las métricas cuantitativas de exactitud, latencia, FPS, memoria e inicialización desarrolladas en la sección 1.1. De esta manera, la selección no depende solamente de que un modelo reconozca objetos, sino de que pueda prestar el servicio de asistencia dentro de las restricciones del dispositivo objetivo.

2.3 Modelos candidatos

A partir de la revisión se definieron cuatro arquitecturas candidatas para la comparación experimental: Faster R-CNN, YOLO, SSD MobileNet y EfficientDet. La selección considera que deben detectar varias clases en una imagen, disponer de implementaciones o pesos reutilizables y permitir una ejecución razonable en dispositivos móviles.

Faster R-CNN representa la alternativa de dos etapas y ofrece alta precisión mediante una Red de Propuesta de Regiones, pero su costo de inferencia y memoria puede limitar el tiempo real en un teléfono. Las estrategias de compresión descritas por Magdy et al. (2025) muestran posibilidades de optimización, aunque el prototipo de este trabajo no entrena ni comprime este modelo.

SSD MobileNet combina detección de una etapa con un backbone ligero. SSD predice objetos en mapas de características de diferentes escalas y MobileNet reduce el costo de las convoluciones (Howard et al., 2017; Liu et al., 2016; Sandler et al., 2018). Esta combinación favorece la velocidad, pero su precisión de localización debe verificarse para las clases y escenas interiores del conjunto propio.

EfficientDet es una familia de detectores de una etapa basada en EfficientNet, una red piramidal de características bidireccional (BiFPN, bidirectional feature pyramid network) y escalamiento compuesto (Tan et al., 2020). La documentación de EfficientDet-Lite describe variantes para dispositivos móviles (Google, s. f.-g); por ello, EfficientDet-Lite2 se considera una opción intermedia entre precisión, tamaño y velocidad.

YOLO realiza la detección en una sola pasada y dispone de variantes con diferentes tamaños, desde modelos pequeños hasta modelos de mayor capacidad (Redmon et al., 2016). Las variantes YOLOv8 y YOLO11 evaluadas se documentan por separado en Ultralytics (Ultralytics, s. f.-a; Ultralytics, s. f.-b); mientras que el tamaño de la variante y la resolución de entrada determinan el costo real de ejecución en Android.

Las clases prioritarias del prototipo se relacionan con categorías disponibles en COCO, conjunto que contiene anotaciones para detección y segmentación en escenas cotidianas (Lin et al., 2014). También se revisó Open Images como fuente de categorías y anotaciones de gran escala (Kuznetsova et al., 2020). En la implementación final se mantuvo COCO porque EfficientDet-Lite2 ya dispone de pesos compatibles con sus categorías y no fue necesario entrenar un modelo propio.

La portabilidad de los modelos también depende del formato de inferencia. TensorFlow Lite permite ejecutar archivos .tflite en el dispositivo (TensorFlow, s. f.), mientras el motor para Open Neural Network Exchange (ONNX Runtime) y ExecuTorch representan alternativas para otros ecosistemas de despliegue (ONNX Runtime, s. f.; PyTorch, s. f.). AssistiveVision utiliza TensorFlow Lite por su integración directa con el modelo EfficientDet-Lite2 seleccionado.

2.4 Entorno experimental

2.4.1 Equipo de escritorio

La primera fase se ejecutó en un equipo con sistema operativo Arch Linux x86_64, procesador Intel Core i5-12450H de 8 núcleos y 12 hilos, gráficos integrados Intel UHD Graphics y 8 gigabytes (GB) de memoria RAM. La inferencia se realizó mediante TensorFlow Lite sobre CPU, con el fin de obtener métricas de exactitud en entorno controlado.

2.4.2 Dispositivo móvil Android

Las pruebas de despliegue se realizaron sobre un dispositivo Xiaomi Redmi Note 13 4G (modelo 23129RA5FL), representativo de la gama media en el mercado colombiano. El dispositivo cuenta con procesador Qualcomm Snapdragon 685 Octa-core a 2.8 GHz, GPU Adreno 610, 8 GB de RAM de bajo consumo de cuarta generación extendida (LPDDR4X, low power double data rate 4X) y opera bajo Android 15.

Se evaluaron dos backends de ejecución: CPU con 4 hilos de procesamiento, y GPU Delegate mediante TFLite GPU Delegate con soporte OpenCL. Se realizaron 50 ejecuciones por modelo para obtener mediciones estables de latencia y rendimiento temporal.

2.5 Metodología de evaluación

2.5.1 Modelos evaluados

Se evaluaron seis configuraciones de modelos, todas en formato TensorFlow Lite y con pesos preentrenados sobre el dataset COCO: YOLOv8s en variantes Float32 y Float16 (Ultralytics, s. f.-a), YOLO11n en variantes Float32 y Float16 (Ultralytics, s. f.-b), SSD MobileNetV1 (Howard et al., 2017; Liu et al., 2016) y EfficientDet-Lite2 (Google, s. f.-g; Tan et al., 2020). La selección de estas configuraciones responde a los criterios definidos en la sección 2.2.

En escritorio se calculó AP mediante pycocotools y precisión, recall y F1 mediante emparejamiento por clase e IoU. El protocolo COCO y el cálculo de métricas a un umbral fijo se documentan por separado para distinguir la evaluación de la curva de puntuaciones de la operación del prototipo (Lin et al., 2014).

Las pruebas en Android se realizaron con la LiteRT Benchmark Tool, específicamente el binario compilado android_aarch64_benchmark_model, ejecutado vía Android Debug Bridge (ADB). Este binario en C++ permite medir el tiempo de inferencia directamente sobre el dispositivo con soporte para aceleración por GPU, sin incluir captura de cámara ni postprocesado de resultados, con un alcance limitado a la ejecución del modelo. Se ejecutaron 50 iteraciones por modelo con 4 hilos de procesamiento (TensorFlow contributors, s. f.).

La ejecución remota del benchmark sobre el dispositivo se realizó mediante Android Debug Bridge (ADB), la herramienta oficial de línea de comandos de la plataforma Android que permite transferir los modelos y el binario al dispositivo, lanzar las mediciones y recopilar los resultados desde el equipo de desarrollo, mediante comandos automatizados (Google, s. f.-a).

2.5.2 Dataset de evaluación

El conjunto inicial reúne imágenes propias de interiores e imágenes externas obtenidas de Roboflow. La exportación del 5 de junio de 2026 declara licencia de atribución Creative Commons 4.0 (CC BY 4.0) y se conserva con las imágenes y anotaciones en el repositorio del proyecto. La anotación manual se realizó en Roboflow (Roboflow, s. f.), generando un total de 80 imágenes con 257 anotaciones en formato COCO JSON (Lin et al., 2014).

Las categorías evaluadas fueron: persona (38 anotaciones), silla (81 anotaciones), sofá (20 anotaciones), cama (16 anotaciones), mesa (29 anotaciones), inodoro (13 anotaciones), televisor (20 anotaciones), computador portátil (13 anotaciones), lavamanos (16 anotaciones) y refrigerador (11 anotaciones), correspondientes a los identificadores COCO (Lin et al., 2014).

Las métricas de precisión fueron calculadas mediante la librería pycocotools, que implementa el protocolo oficial de evaluación COCO (COCO dataset contributors, s. f.; Lin et al., 2014).

2.6 Resultados

2.6.1 Resultados en equipo de escritorio

La Tabla 2.1 presenta las métricas de exactitud obtenidas en el equipo de escritorio mediante TensorFlow Lite sobre CPU: mAP50, mAP50-95, precisión, recall y F1.

Tabla 2.1. Resultados de evaluación en equipo de escritorio.

Modelo	mAP50	mAP50-95	Precisión	Recall	F1
YOLOv8s Float32	0.642	0.514	0.733	0.673	0.702
YOLOv8s Float16	0.642	0.514	0.733	0.673	0.702
YOLO11n Float32	0.615	0.498	0.723	0.619	0.667
YOLO11n Float16	0.614	0.498	0.718	0.615	0.662
SSD MobileNetV1	0.518	0.336	0.343	0.545	0.421
EfficientDet-Lite2	0.648	0.486	0.713	0.677	0.695

Fuente: elaboración propia con datos y especificaciones del proyecto.

La Tabla 2.2 presenta el desglose de AP50 por cada clase del conjunto de evaluación.

Tabla 2.2. Desglose de AP50 por clase para cada modelo.

Clase	YOLOv8s F32	YOLOv8s F16	YOLO11n F32	YOLO11n F16	SSD MobileNet V1	EfficientDet Lite2
person	0.731	0.731	0.617	0.617	0.534	0.770
chair	0.580	0.580	0.464	0.456	0.328	0.546
couch	0.662	0.662	0.647	0.647	0.621	0.799
bed	0.709	0.709	0.719	0.719	0.632	0.812
dining table	0.362	0.366	0.343	0.339	0.247	0.316
toilet	0.921	0.921	0.921	0.921	0.798	0.915
tv	0.472	0.472	0.628	0.628	0.596	0.592
laptop	0.350	0.350	0.333	0.333	0.153	0.284
sink	0.743	0.743	0.665	0.665	0.463	0.631
refrigerator	0.886	0.886	0.812	0.812	0.812	0.812

Fuente: elaboración propia con datos y especificaciones del proyecto.

Las Tablas 2.1 y 2.2 permiten distinguir el desempeño agregado de las diferencias por clase. EfficientDet-Lite2 obtuvo la mayor mAP50 (0,648), mientras que YOLOv8s alcanzó la mayor mAP50-95 (0,514) y F1 (0,702). EfficientDet-Lite2 destacó en persona, sofá y cama, pero YOLOv8s obtuvo mejores resultados en mesa de comedor y computador portátil. Por tanto, la ventaja agregada de un detector no implica que sea el mejor en todas las categorías; esta comparación se complementa con su costo de ejecución en Android.

2.6.2 Resultados en Android — CPU

La Tabla 2.3 presenta las métricas correspondientes a la ejecución de los modelos en el dispositivo móvil utilizando CPU con 4 hilos de procesamiento. Se reportan FPS, latencia promedio, mediana y rango mínimo/máximo para reflejar la estabilidad de cada modelo, así como el tiempo de inicialización y el consumo de memoria RAM.

Tabla 2.3. Resultados de *benchmark* en Android utilizando CPU.

Modelo	FPS	Init (ms)	RAM (MB)	Lat. media	Lat. mediana	Lat. mín	Lat. máx
YOLOv8s Float32	1.61	393.2	164.1	620.8 ms	621.0 ms	612.0 ms	627.8 ms
YOLOv8s Float16	1.61	364.9	150.8	622.1 ms	622.1 ms	620.3 ms	627.3 ms
YOLO11n Float32	5.30	220.7	80.5	188.6 ms	188.5 ms	187.0 ms	193.6 ms
YOLO11n Float16	5.05	144.0	79.8	198.1 ms	197.3 ms	188.7 ms	220.3 ms
SSD MobileNetV1	42.75	57.7	18.5	23.4 ms	23.4 ms	23.3 ms	23.5 ms
EfficientDet-Lite2	8.00	122.7	65.8	124.9 ms	124.9 ms	124.5 ms	125.5 ms

Fuente: elaboración propia con datos y especificaciones del proyecto.

2.6.3 Resultados en Android — GPU Delegate

Se evaluó el uso de GPU Delegate sobre los modelos compatibles. SSD MobileNetV1 fue excluido de esta evaluación por no presentar soporte estable con el backend GPU en el dispositivo utilizado. La Tabla 2.4 reporta FPS, latencia promedio y su diferencia porcentual respecto a CPU, así como el tiempo de inicialización y el consumo de memoria RAM.

Tabla 2.4. Resultados de *benchmark* en Android utilizando GPU Delegate.

Modelo	FPS	Init (ms)	RAM (MB)	Lat. promedio	vs CPU
YOLOv8s Float32	1.34	4018.4	188.3	743.8 ms	19.8% más lento
YOLOv8s Float16	1.35	3001.7	167.9	741.1 ms	19.1% más lento
YOLO11n Float32	3.85	4351.4	96.8	259.6 ms	37.6% más lento
YOLO11n Float16	3.90	3618.7	92.6	256.6 ms	29.5% más lento
EfficientDet-Lite2	2.79	3019.0	147.1	357.9 ms	186.5% más lento

Fuente: elaboración propia con datos y especificaciones del proyecto.

Las Tablas 2.3 y 2.4 muestran que el delegado GPU no redujo la latencia de ninguno de los modelos probados. En EfficientDet-Lite2, el promedio pasó de 124,9 ms en CPU a 357,9 ms con GPU; en YOLOv8s Float32 pasó de 620,8 a 743,8 ms. El menor tiempo en CPU correspondió a SSD MobileNetV1 (23,4 ms), aunque sus métricas de detección fueron inferiores en la Tabla 2.1. Estos resultados motivan el análisis conjunto de exactitud, latencia y memoria de la sección 2.7 y la selección final de la sección 2.8.

2.7 Análisis de resultados

2.7.1 Precisión vs. costo computacional

El uso previsto de AssistiveVision hace que la latencia tenga un peso decisivo en la selección. Cada captura solicitada por el usuario debe procesarse antes de generar el mensaje de voz, de modo que el tiempo de inferencia contribuye a la espera y al desfase entre la escena registrada y su descripción. Se busca reducir esa espera conservando capacidad para identificar los objetos del entorno. Por ello, las métricas de exactitud se interpretan junto con el tiempo de procesamiento y la memoria que exige cada modelo en el teléfono.

YOLOv8s obtuvo el mayor mAP50-95 (0,514) del conjunto evaluado. Esta métrica combina coincidencia de clase y solapamiento de cajas; no mide directamente la exactitud de la zona anunciada. Su latencia aislada en Android CPU supera los 620 ms en ambas variantes, equivalente a menos de dos inferencias por segundo, lo que incrementa el costo temporal del componente de detección en el hardware evaluado.

SSD MobileNetV1 obtuvo la menor latencia aislada (23,4 ms, 42,75 inferencias/s) y el menor mAP50-95 (0,336) del conjunto inicial. Estos resultados describen un compromiso entre velocidad y ajuste de las cajas; su efecto sobre descriptores espaciales requiere una evaluación específica de la localización.

Con este criterio, EfficientDet-Lite2 ofrece un equilibrio favorable. Alcanzó el mayor mAP50 del conjunto (0,648) y una latencia de inferencia de 124,9 ms en Android CPU (8,00 inferencias/s), frente a 620,8 ms de YOLOv8s Float32: aproximadamente una quinta parte del tiempo. Aunque su mAP50-95 fue menor (0,486 frente a 0,514), las puntuaciones F1 fueron próximas (0,695 y 0,702). Se aceptó, por tanto, un menor ajuste de las cajas a umbrales de solapamiento exigentes para reducir el tiempo del componente de detección, conservando una F1 cercana a la de YOLOv8s en esta evaluación. Estas latencias corresponden a inferencia aislada; el ciclo de captura y voz incorpora etapas adicionales.

YOLO11n alcanzó 5.05–5.30 FPS en Android CPU con latencias entre 188.6 y 198.1 ms, con un mAP50-95 de 0.498.

En el conjunto inicial, YOLOv8s y EfficientDet-Lite2 alcanzaron F1 de 0,702 y 0,695. SSD MobileNetV1 obtuvo precisión de 0,343 y *recall* de 0,545, lo que implica abundantes predicciones falsas y objetos sin recuperar a ese umbral. YOLO11n Float32 obtuvo precisión de 0,723 y *recall* de 0,619. Estas cifras corresponden a detecciones en imágenes; no representan la tasa de anuncios de voz, que también depende del filtrado y la selección de objetos.

2.7.2 Float32 vs. Float16

Float16 no ofreció una mejora apreciable de velocidad en las mediciones del Redmi Note 13: las diferencias de YOLO11n entre variantes fueron inferiores a 10 ms. Las métricas de exactitud también fueron próximas. Se observó una reducción del tamaño de archivo, pero el experimento no permite atribuir las diferencias a una limitación específica de las unidades aritméticas del procesador o de la GPU.

2.7.3 CPU vs. GPU Delegate

La CPU obtuvo mayor velocidad que GPU Delegate en los modelos medidos en el Redmi Note 13: YOLO11n Float32 pasó de 5,30 a 3,85 inferencias/s; EfficientDet-Lite2, de 8,00 a 2,79; y YOLOv8s Float32, de 1,61 a 1,34. Esta observación sustenta el uso de CPU para esa configuración. El mecanismo causal exacto de la pérdida de rendimiento no se aisló y no debe generalizarse a otras GPU o teléfonos.

El análisis de los logs del benchmark tool muestra diferencias de inicialización y memoria entre CPU y GPU. La primera es el tiempo de inicialización: todos los modelos requieren entre 3002 ms y 4351 ms para inicializar la sesión con GPU Delegate, frente a tiempos significativamente menores en CPU. Aunque este overhead no afecta la latencia de inferencia por frame una vez inicializado, es compatible con los costes de preparación del delegado descritos en la documentación (Google, s. f.-e); no se midieron por separado compilación, transferencias o sincronización.

El consumo de memoria RAM refuerza esta conclusión: las variantes con GPU Delegate requirieron entre 12 MB y 81 MB adicionales respecto a CPU para el mismo modelo (por ejemplo, YOLOv8s Float32 pasó de 164.1 MB en CPU a 188.3 MB en GPU), sin que esta medición permita identificar qué asignaciones explican la diferencia. En conjunto, SSD MobileNetV1 presentó el menor consumo absoluto (18.5 MB), seguido de EfficientDet-Lite2 (65.8 MB) y YOLO11n (aproximadamente 80 MB), mientras que YOLOv8s superó los 150

MB en todas sus variantes. Asimismo, el tiempo de inicialización con GPU Delegate osciló entre 3002 ms y 4351 ms, frente a un rango de 58 a 393 ms en CPU, lo que implica un retraso perceptible de varios segundos desde la apertura de la aplicación hasta que el sistema estaría operativo.

2.8 Selección del modelo final

Con base en los resultados experimentales, la Tabla 2.5 presenta la comparación directa entre los tres modelos que presentaron mejor equilibrio general.

Tabla 2.5. Comparación de candidatos finales.

Modelo	mAP50	mAP50-95	FPS Android	Tamaño	Ventaja
YOLO11n Float32	0.615	0.498	5.30	10 MB	Mejor mAP50-95
YOLO11n Float16	0.614	0.498	5.05	5.2 MB	Menor tamaño
EfficientDet-Lite2	0.648	0.486	8.00	7.2 MB	Mayor FPS y mAP50

Fuente: elaboración propia con datos y especificaciones del proyecto.

La reducción a Float16 fue descartada dado que no ofrece ventaja de velocidad en el hardware evaluado, y la diferencia de tamaño (4,8 MB) no representa un factor determinante para la distribución de la aplicación. En cambio, la latencia de inferencia resulta crítica para la experiencia de asistencia: como la aplicación congela la captura y solo emite la descripción de voz cuando termina la detección, un mayor tiempo de inferencia retrasa directamente la respuesta auditiva y aumenta el desfase entre el instante en que el usuario apunta la cámara y el momento en que escucha qué objetos tiene enfrente.

Se seleccionó EfficientDet-Lite2 en CPU porque combinó el mayor mAP50 del conjunto (0,648) y un recall de 0,677 con un costo temporal menor que los candidatos YOLO. La latencia fue decisiva para el propósito de ofrecer descripciones auditivas oportunas: frente a YOLOv8s Float32, el modelo requirió aproximadamente una quinta parte del tiempo de inferencia y menos memoria (65,8 frente a 164,1 MB). También superó a YOLO11n Float32 en mAP50 (0,648 frente a 0,615), latencia (124,9 frente a 188,6 ms) y memoria (65,8 frente a 80,5 MB). Se aceptó una mAP50-95 inferior a la de ambos modelos para priorizar el tiempo de procesamiento sin renunciar a la recuperación de objetos observada. La elección se circunscribe al conjunto y al teléfono evaluados; la implementación y la evaluación posterior se presentan en los capítulos 3 y 4.

3. Diseño e implementación del prototipo AssistiveVision

Este capítulo describe el diseño y la implementación del prototipo AssistiveVision, la aplicación móvil para Android que materializa el sistema de asistencia visual. Se presenta primero la arquitectura general del sistema y luego el detalle de cada uno de sus módulos, junto con las decisiones de implementación más relevantes.

3.1 Arquitectura general del sistema

El flujo actual comprende captura, preprocesamiento, detección, selección por confianza y número máximo de objetos, localización en la imagen y generación de voz. La Figura 3.1 representa el orden ejecutado por MainViewModel. La inversión del letterbox devuelve las cajas al fotograma original antes de asignar las zonas. Los módulos de inferencia y audio se inicializan una vez y se liberan al finalizar su ciclo de vida. La implementación de estos módulos se conserva en la revisión del código fuente citada en (Cerda Leon, 2026).

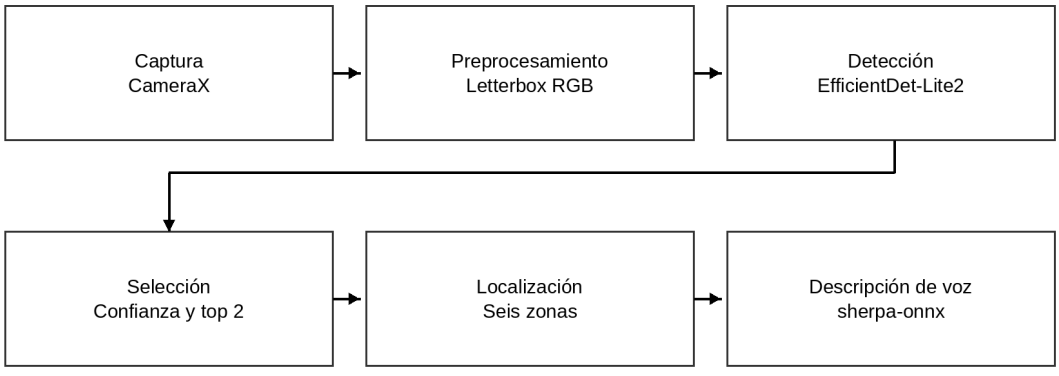


Figura 3.1. Flujo de procesamiento de la implementación actual.

Fuente: elaboración propia a partir del diseño y los resultados del proyecto.

3.2 Posicionamiento del dispositivo

Se compararon arnés de cabeza, arnés de pecho y palo de selfie extensible con patas plegables de trípode. Este soporte permite al usuario orientar libremente el celular y elegir qué parte de la escena explorar. En las capturas de este estudio se apuntó la cámara hacia los objetos de interés. Las capturas de evaluación son verticales y combinan dos campos visuales nominales, 1x y 0,5x. La altura y la inclinación no se registraron como mediciones experimentales, por lo que no se prescribe un ángulo o una altura óptima. El capítulo 4 compara su cobertura y detecciones a partir de los grupos de fotografías disponibles. El campo visual de la cámara es la extensión angular de la escena registrada; depende de la óptica, el sensor y el recorte y no equivale al campo visual fisiológico del usuario. Las etiquetas 1x y 0,5x no son ángulos calibrados ni permiten afirmar que el campo angular se duplica. En las tablas y figuras se abrevia como «palo de selfie». La Figura 3.2 presenta los tres tipos de soporte utilizados; la Figura 4.2 ilustra su colocación para la captura.

Figura 3.2. Tipos de soporte utilizados para posicionar el dispositivo.



(a) Cabeza (b) Pecho (c) Palo de selfie

Fuente: fichas de los soportes de cabeza; pecho; palo de selfie (diseño de referencia).

3.3 Módulo de captura

CameraX 1.6.1 gestiona la cámara y su ciclo de vida (Google, s. f.-c). Preview mantiene la vista de la escena e ImageAnalysis entrega fotogramas RGBA_8888 (rojo, verde, azul y alfa, con ocho bits por canal) con STRATEGY_KEEP_ONLY_LATEST. FrameStore conserva una copia utilizable del fotograma más reciente. La política de selección de cámara distingue la cámara trasera principal de alternativas de campo amplio, sujetas a

las capacidades del dispositivo; el 0,5x del conjunto iPhone no demuestra una focal equivalente en Android.

La inferencia se ejecuta bajo demanda. Al pulsar Capturar fotograma, la aplicación copia el último fotograma disponible y congela la vista mientras procesa la escena y reproduce la voz. No vuelve a aceptar una captura mientras la anterior está activa. El fin del audio o un error desbloquean la vista; un temporizador de respaldo de 20 s evita que quede congelada indefinidamente.

3.4 Módulo de preprocesamiento

El módulo de preprocesamiento transforma los fotogramas de la cámara al formato de tensor que espera EfficientDet-Lite2: un tensor de enteros sin signo de 8 bits con dimensiones [1, 448, 448, 3] en orden de canales rojo, verde y azul (RGB, red, green, blue). Cada fotograma se redimensiona aplicando la técnica de letterbox, es decir, se escala manteniendo la relación de aspecto y se completa con un relleno gris (114, 114, 114) hasta alcanzar 448×448 píxeles, evitando la distorsión geométrica de la escena. No se aplica normalización a valores flotantes, de acuerdo con la especificación de entrada del modelo (Cerda Leon, 2026). En [1, 448, 448, 3], los ejes representan lote, alto, ancho y canales.

3.5 Módulo de detección

DetectionModule utiliza la API Interpreter del paquete LiteRT 1.4.2 y el archivo efficientdet-lite2-detection-metadata.tflite, cargado mediante un MappedByteBuffer (Google, s. f.-f). La inferencia se ejecuta en CPU. Se leen cajas normalizadas, clases, puntuaciones y número de detecciones; el intérprete valida la forma del tensor de cajas al inicializarse.

Los índices de clase emitidos por el modelo se desplazan en una unidad para obtener los identificadores del dataset COCO, y se conservan únicamente las diez clases prioritarias del sistema, listadas en la Tabla 3.1. Las detecciones se filtran por un umbral de confianza configurable (0.30 por defecto, ajustable entre 0.10 y 0.90), se ordenan por puntuación descendente y se conservan como máximo quince detecciones por fotograma.

Tabla 3.1. Clases de objetos prioritarias anunciadas por el sistema.

Identificador COCO	Clase (inglés)	Descriptor en español
1	person	persona
62	chair	silla
63	couch	sofá
65	bed	cama
67	dining table	mesa
70	toilet	inodoro
72	tv	televisor
73	laptop	computadora
81	sink	lavabo
82	refrigerator	refrigerador

Fuente: elaboración propia con datos y especificaciones del proyecto.

3.6 Módulo de localización espacial

LocalizationModule asigna seis zonas según el centro de la caja: cortes en 0,35 y 0,65 del ancho y a media altura. Son regiones de imagen, no distancias ni tercios exactos (Tabla 3.2 y Figura 3.3). Para una caja [x, y, w, h], el centro es (cx, cy) = (x + w/2, y + h/2). Si W y H son el ancho y alto del fotograma original, u = cx/W y v = cy/H son sus coordenadas normalizadas. El origen está arriba a la izquierda; x crece a la derecha e y hacia abajo. Izquierda corresponde a $u < 0{,}35$, centro a $0{,}35 \leq u < 0{,}65$ y derecha a $u \geq 0{,}65$; la mitad superior cumple $v < 0{,}50$.

Las cajas del modelo usan coordenadas normalizadas [ymin, xmin, ymax, xmax] sobre la imagen con relleno. Esa representación se convierte al fotograma original antes de aplicar las reglas anteriores y difiere del formato [x, y, w, h] empleado en las anotaciones. El descriptor «frente a ti» corresponde a la región inferior central y no acredita ausencia de obstáculos o distancia segura. Los límites descritos corresponden a LocalizationModule en la revisión evaluada (Cerda Leon, 2026).

Tabla 3.2. Zonas de localización espacial y descriptores asociados.

Zona	Descriptor (español)	Descriptor (inglés)
Inferior izquierda	abajo a la izquierda	lower left
Inferior centro	frente a ti	in front of you
Inferior derecha	abajo a la derecha	lower right
Superior izquierda	arriba a la izquierda	upper left
Superior centro	arriba al centro	centered above
Superior derecha	arriba a la derecha	upper right

Fuente: elaboración propia con datos y especificaciones del proyecto.



Figura 3.3. Regiones de la imagen para la localización relativa.

Fuente: elaboración propia a partir del diseño y los resultados del proyecto.

3.7 Módulo de filtrado y priorización

La implementación actual filtra por confianza, ordena las detecciones y limita la lista a uno, dos o tres objetos, con dos por defecto. La captura manual regula cuándo se procesa y se anuncia la escena. El antiguo filtro por período de enfriamiento no participa en este flujo; describirlo como una función activa introduciría una diferencia entre el informe y el código evaluado.

La prioridad de las diez clases de la Tabla 3.1 responde a una delimitación técnica para explorar mobiliario, equipamiento doméstico y presencia de personas, junto con la disponibilidad de etiquetas COCO (Lin et al., 2014). No procede de un estudio de necesidades con usuarios ni establece una jerarquía clínica o de riesgo. La propuesta

menciona puertas, sillas y escritorios como ejemplos (Cerda Leon, s. f.); la clase silla está incluida, pero el vocabulario COCO empleado no contiene una clase puerta ni una clase independiente escritorio. La etiqueta dining table corresponde a mesa de comedor y no permite afirmar detección general de escritorios. Open Images se consideró como alternativa de vocabulario (Kuznetsova et al., 2020), pero no se entrenó ni evaluó aquí un detector con ese conjunto. Incorporar puertas exige seleccionar o adaptar un modelo compatible, anotar ejemplos y evaluar su desempeño.

3.8 Módulo de audio y descriptores de voz

AudioModule utiliza sherpa-onnx v1.13.6 para sintetizar voz local a partir de modelos Variational Inference with adversarial learning for end-to-end Text-to-Speech (VITS) y Piper empaquetados con la aplicación (k2-fsa, s. f.; Kim et al., 2021; Rhasspy contributors, s. f.). El catálogo incluye es_MX-claude-high-int8 para español y en_US-amy-low-int8 para inglés, junto con los datos de fonemización necesarios. La implementación actual no depende del servicio TextToSpeech de Android ni de una solicitud de síntesis por red.

SpeechFormatter construye una oración con artículos y conectores localizados. Por ejemplo: «Hay una persona donde estás apuntando y una silla abajo a la izquierda de donde estás apuntando». Las voces y la velocidad se seleccionan según las preferencias. La reproducción se realiza mediante AudioTrack (Google, s. f.-b); la terminación y los errores se comunican al flujo de captura para liberar el fotograma congelado. La verificación de la composición gramatical y de la síntesis local se detalla en la sección 4.9.

El diseño de los descriptores combina presencia, nombre del objeto y región relativa al punto de orientación de la cámara. La redacción localizada y el límite de dos objetos por defecto garantizan mensajes breves, estructurados y libres de saturación informativa. Las pruebas de SpeechFormatter verifican la composición gramatical y espacial, mientras la evaluación de hasta dos objetos cuantifica la precisión de la selección entregada al módulo de voz.

3.9 Modos de operación

El modo evaluado es manual: el usuario activa Capturar fotograma y recibe una descripción de los objetos seleccionados. La Figura 3.4 muestra los estados principales de la interacción. La evaluación de imágenes reproduce el procesamiento de detección, pero no mide la interacción completa ni el consumo energético.

La exploración continua por voz se plantea como ampliación futura. El código y los resultados de este informe no sustentan una frecuencia de análisis automático ni una evaluación de usabilidad de ese modo.

3.10 Configuración del sistema

La aplicación incluye una pantalla de configuración que permite ajustar el comportamiento del sistema sin necesidad de reiniciarla, almacenando las preferencias con Jetpack DataStore (Google, s. f.-d). La Tabla 3.3 resume los parámetros disponibles.

Tabla 3.3. Parámetros configurables del sistema.

Parámetro	Rango	Valor por defecto
Idioma de voz e interfaz	español / inglés	español
Velocidad de habla	0.5× – 2.5×	1.0×
Umbral de confianza	0.10 – 0.90	0.30
Máximo de objetos anunciados	1, 2 o 3	2
Mostrar vista previa de cámara	sí / no	sí

Fuente: elaboración propia con datos y especificaciones del proyecto.

La opción de mostrar la vista previa controla la presentación de la imagen. No se midió su efecto sobre consumo energético o privacidad percibida, por lo que esos beneficios no se presentan como resultados experimentales.

3.11 Interfaz de usuario

La interfaz incluye vista previa, captura manual y acceso a configuración, con textos localizados. Durante el procesamiento conserva el fotograma seleccionado y muestra las detecciones y su información espacial. La secuencia de estados se resume en la Figura 3.4; los controles y textos concretos se encuentran en los recursos del proyecto Android. La Figura 3.5 muestra las pantallas de captura y de configuración de la aplicación.

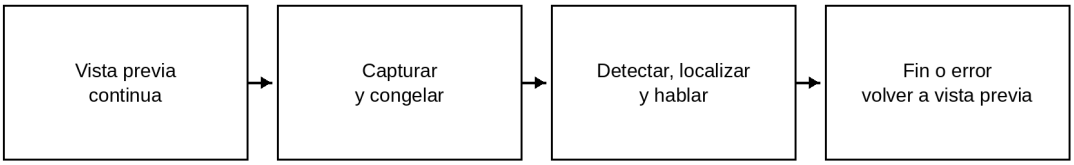


Figura 3.4. Secuencia de interacción manual y liberación de la captura.

Fuente: elaboración propia a partir del diseño y los resultados del proyecto.

Figura 3.5. Pantallas de captura y configuración de AssistiveVision.

(a) Captura manual



(b) Configuración



Fuente: capturas del autor de AssistiveVision en un emulador Android, en español, con confianza 0,30 y hasta dos objetos. La escena virtual de la vista previa ilustra la interfaz y no pertenece al conjunto experimental.

La interacción para personas ciegas se apoya en la salida de voz y en los mecanismos de accesibilidad de Android. Los controles Capturar y Ajustes son enfocables y exponen el rol de botón al lector de pantalla TalkBack; los iconos decorativos se excluyen de su recorrido, y los controles de volver y de pantalla completa tienen descripciones de contenido. El estado textual se configura como una región de actualización cortés para que el lector de pantalla pueda anunciar sus cambios. Así, la información esencial de la captura no depende exclusivamente de las cajas y etiquetas dibujadas sobre la vista previa.

Además del control táctil, una tecla física compatible o un disparador Bluetooth puede solicitar la captura desde la pantalla de cámara. La descripción sintetizada comunica los objetos seleccionados y su región relativa, mientras que los ajustes permiten cambiar idioma, velocidad de voz y número máximo de objetos. La interacción se apoya así en la voz, los controles identificables y la captura física. Como continuación del trabajo se propone evaluar la usabilidad con personas ciegas y el recorrido completo de la interfaz con TalkBack.

3.12 Verificación y rendimiento del prototipo

La revisión actual se verificó mediante 34 pruebas de la máquina virtual de Java (JVM, Java Virtual Machine) sin fallos: localización (6), geometría letterbox (6), formato de descriptores (7), política de lentes (5), catálogo de voces (7), tecla de captura (2) y una prueba básica (1). Además, la reproducción de las 198 imágenes y cinco pruebas de síntesis de voz y gestión de fotogramas en modo avión completaron la verificación instrumentada en Android. Ocho pruebas del evaluador y de la estadística verifican emparejamiento, casos vacíos, bootstrap y cálculo de p-valores.

El *benchmark* aislado de EfficientDet-Lite2 en el Redmi Note 13 registró 124,9 ms (capítulo 2). La reproducción posterior en emulador obtuvo una mediana de 246,3 ms para el método de detección y 104,0 ms para decodificación y preprocesamiento. Estos alcances son distintos y no se suman sus medianas para estimar una mediana total. La cifra histórica de 137,8 ms incluida en el README de la aplicación no dispone aquí de un registro comparable del flujo completo; no se usa como medición validada del sistema actual ni se equipara a 8,55 FPS.

3.13 Requerimientos y criterios de verificación

La Tabla 3.4 explicita los requerimientos operativos que se desprenden de la propuesta y de las decisiones implementadas. Es una sistematización de la especificación actual; no representa una sesión de levantamiento de necesidades con usuarios. Se distingue la verificación técnica disponible de los criterios de aceptación que aún requieren ensayos.

Tabla 3.4. Requerimientos operativos y evidencia de verificación.

Requerimiento	Criterio verificable	Evidencia y alcance
Captura bajo demanda	Procesar una captura y liberar la vista al terminar o fallar	CameraX y flujo manual, secciones 3.3 y 3.11; falta prueba de interacción con usuarios.
Detección local	Ejecutar el modelo seleccionado y conservar cajas, clases y puntuaciones	EfficientDet-Lite2 y reproducción de 198 imágenes, sección 3.5 y capítulo 4.
Ubicación relativa	Asignar una de seis zonas al centro de la caja	LocalizationModule y seis pruebas JVM; no demuestra precisión de ubicación física.
Descripción auditiva	Construir mensajes localizados y sintetizar voz en el dispositivo	SpeechFormatter, AudioModule, catálogo de voces y sección 4.9 (14 pruebas JVM y 5 instrumentadas en modo avión).
Salida configurable	Aplicar confianza y máximo de objetos elegidos	Lista de uno a tres objetos, dos por defecto; análisis de selección en sección 4.5.
Rendimiento y montaje	Medir tiempos con alcance explícito y comparar seis configuraciones	Capítulos 2 y 4; faltan latencia completa, ergonomía y estabilidad en uso.

Fuente: elaboración propia con datos y especificaciones del proyecto.

El cumplimiento de los requerimientos se verifica mediante la exactitud de la selección anunciada (sección 4.5), la distribución de tiempos de procesamiento (sección 4.7) y las pruebas unitarias e instrumentadas de composición gramatical y síntesis local de voz (sección 4.9).

4. Evaluación técnica del sistema y de las condiciones de captura

4.1 Diseño y protocolo de evaluación

La evaluación aborda tres preguntas con un mismo archivo de fotografías: cómo varían las instancias visibles y correctamente detectadas entre montajes; qué cambia al combinar montaje y lente; y cómo se distribuye el recall en bandas de distancia estimada. Estas preguntas son líneas de análisis, no tres espacios físicos ni tres conjuntos nuevos. Primero se definen las unidades del estudio y se describen los espacios físicos; después se presenta la reproducción de las imágenes en Android y se interpretan los resultados. El cierre integra la evaluación de la retroalimentación auditiva en sus dimensiones informativa, sintáctica y de síntesis local.

4.1.1 Espacios físicos y contenido de las capturas

El análisis se organiza a partir de seis unidades de observación. Un inmueble es la edificación o vivienda donde se realiza la toma; un espacio físico (o recinto) es una habitación o zona real dentro de él; una escena es la disposición observable desde una orientación de la cámara; un grupo de captura (identificado con los códigos G01 a G33) reúne seis fotografías sucesivas tomadas en una misma ubicación para comparar las seis configuraciones de montaje y lente; y una fotografía es cada una de esas observaciones (33 × 6 = 198 fotografías en total, cifra que no equivale a 33 habitaciones distintas). Por su parte, una instancia corresponde a cada aparición anotada de un objeto dentro de una fotografía, de modo que un mismo objeto físico puede aportar varias instancias entre vistas.

Las 198 fotografías se adquirieron en seis inmuebles (cinco viviendas y un edificio de oficinas), que abarcan 30 recintos físicos únicos distribuidos en los 33 grupos G01 a G33:

Vivienda 1 (dormitorios G01, G02 y G05; sala-comedor G03; cocina G04; baño G06), Vivienda 2 (sala-comedor G07; cocina G08; baño G09), Vivienda 3 (sala-comedor en dos orientaciones, G10 y G12; cocina principal G11; dormitorios G13, G15–G16 en dos orientaciones y G18; baños G14, G17, G19 y G21; sala de estar G20; cocina auxiliar G22), Vivienda 4 (sala-comedor G23; cocina G24; dormitorio G25; baño G26), edificio de oficinas (sala de trabajo en dos sectores, G27 y G30; terraza cubierta G28; recepción G29) y Vivienda 5 (dormitorio G31; baño G32; sala-comedor con cocina G33). Los recintos se eligieron por accesibilidad y por contener de forma natural las diez clases objetivo bajo su iluminación habitual (luz diurna y eléctrica en G01–G30; artificial nocturna en G31–G33), sin controlar niveles de iluminancia ni reubicar objetos para inducir oclusiones artificiales. Según su contenido visible, los recintos se agrupan en seis tipologías observables (dormitorios, baños, cocinas, salas o comedores, espacios de trabajo y terraza), con mobiliario cercano, oclusiones parciales, superficies reflectantes y agrupaciones de sillas; la Tabla 4.1 relaciona ejemplos de cada tipología con sus clases anotadas.

Tabla 4.1. Tipologías observables y ejemplos de grupos.

Tipología observable	Grupos de ejemplo	Clases anotadas en los ejemplos
Dormitorio	G01, G02, G13, G31	Cama, silla, mesa, portátil y televisor, según el grupo.
Baño	G06, G09, G14, G32	Inodoro y lavamanos.
Cocina	G04, G08, G11, G24	Refrigerador, lavamanos y, en algunos encuadres, silla o mesa.
Sala o comedor	G03, G07, G12, G20, G33	Silla, mesa y, según el grupo, sofá, televisor, refrigerador o lavamanos.
Espacio de trabajo	G27, G30	Persona, silla, portátil y televisor.
Terraza o zona abierta	G28	Persona, silla y mesa.

Nota. Las categorías provienen de labels.json para las seis fotografías de cada grupo citado (G01 a G33); una clase puede estar visible solo en algunas vistas y las seis capturas del grupo no implican un encuadre idéntico.

La Figura 4.1 presenta ejemplos del conjunto fotográfico.

Figura 4.1. Ejemplos de espacios y condiciones de captura.

(a) Dormitorio



G01 · IMG_3073.jpeg

(b) Sala y comedor



G03 · IMG_3086.jpeg

(c) Cocina



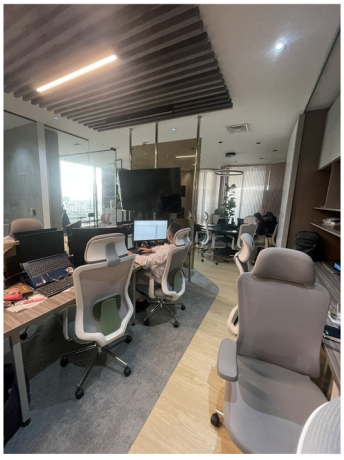
G04 · IMG_3092.jpeg

(d) Baño



G06 · IMG_3104.jpeg

(e) Espacio de trabajo



G27 · IMG_3371.jpeg

(f) Terraza



G28 · IMG_3377.jpeg

Fuente: fotografías del autor conservadas en Build/Testing/data/. Se muestran ejemplos de los grupos G01, G03, G04, G06, G27 y G28, con la imagen completa y sin superponer detecciones. Las etiquetas identifican el grupo y el archivo; no cuentan recintos físicos únicos.

4.1.2 Grupos de captura y configuraciones

Dentro de cada grupo de captura (G01 a G33), las seis fotografías sucesivas corresponden a las seis configuraciones experimentales (c = 1 a 6), obtenidas al cruzar tres montajes del dispositivo (arnés de cabeza, arnés de pecho y palo de selfie dirigido hacia los objetos)

con dos lentes nominales (1x y 0,5x). Como existen 33 grupos, cada configuración reúne 33 fotografías y cada posición de montaje agrupa 66 fotografías (33 grupos × 2 lentes), según el orden detallado en la Tabla 4.2.

Tabla 4.2. Orden y tamaño de las configuraciones de captura.

Orden en el grupo	Montaje	Lente nominal	Fotografías
1	Arnés de cabeza	1x	33
2	Arnés de cabeza	0,5x	33
3	Arnés de pecho	1x	33
4	Arnés de pecho	0,5x	33
5	Palo de selfie dirigido a objetos	1x	33
6	Palo de selfie dirigido a objetos	0,5x	33

El emparejamiento de capturas significa que las seis configuraciones de un grupo se refieren a una situación de captura próxima, de modo que sus diferencias se analizan dentro del grupo. El orden de la Tabla 4.2 se reconstruye a partir de los nombres de archivo y se conserva con estado provisional en capture_groups.csv. No garantiza campos visuales idénticos, objetos inmóviles ni equivalencia completa de la escena: cambiar soporte y lente cambia lo que entra en la imagen. El emparejamiento reduce la comparación entre contextos ajenos, pero no elimina esas diferencias.

Las opciones nominales 1x y 0,5x corresponden a la cámara principal y al ultra gran angular del teléfono de captura. El archivo registra equivalencias focales cercanas a 25 y 14 mm, pero no una calibración del ángulo de campo; además, se identificaron discrepancias entre seis etiquetas nominales y los metadatos del formato intercambiable de archivos de imagen (EXIF, exchangeable image file format). Una vista más amplia puede incorporar más objetos y representar cada uno con menos píxeles en la entrada de 448 × 448. Esa relación geométrica ayuda a interpretar los resultados, sin atribuirle por sí sola la causa de cada error ni suponer que el 0,5x duplica el ángulo de visión.

La Figura 4.2 ilustra la colocación de los tres soportes para la captura; sus tipos se presentan en la Figura 3.2.

Figura 4.2. Ilustración de la colocación de los soportes.



(a) Cabeza (b) Pecho (c) Palo de selfie

Fuente: fotografías del autor.

4.1.3 Reproducción en Android y emparejamiento de cajas

La prueba de instrumentación carga los archivos fotográficos e invoca los módulos de producción de preprocesamiento, detección y transformación inversa de coordenadas. No vuelve a fotografiar los espacios con la cámara Android. EfficientDet-Lite2 recibe 448 × 448 píxeles; cada imagen vertical se escala a 336 × 448 y se completa con gris 114. Se conservan candidatos desde puntuación 0,001 y hasta 15 detecciones para calcular mAP. La evaluación operativa aplica puntuación ≥ 0,30, coincidencia de clase e IoU ≥ 0,50. Las 198 imágenes se recorren en orden barajado con semilla 20261013, después de tres pasadas de calentamiento.

Este emparejamiento de cajas es distinto del emparejamiento de capturas: en cada fotografía se ordenan las predicciones por puntuación y se asigna cada una a una anotación todavía libre de la misma clase cuando alcanza el IoU requerido. Una caja de referencia solo puede contarse una vez. La comparación estadística empareja grupos de captura, mientras que la evaluación del detector empareja cajas dentro de cada imagen, por lo que ambas operaciones actúan sobre niveles de análisis distintos.

4.1.4 Anotaciones y variables de respuesta

Las 198 fotografías proceden de un iPhone 12 Pro Max, con resolución de 1536 × 2048 y orientación vertical incorporada al archivo. Hay 889 instancias anotadas: persona (56), silla (438), sofá (11), cama (56), mesa de comedor (107), inodoro (38), televisor (50), computador portátil (34), lavamanos (56) y refrigerador (43). Dieciséis imágenes no contienen anotaciones de las diez clases y se conservan. El autor corrigió en Roboflow el borrador asistido de cajas y clases; trece cajas se recortaron a los límites por desviaciones mínimas de redondeo. La referencia final es labels.json. La integridad de los archivos se controla mediante sumas de verificación SHA-256, del algoritmo Secure Hash Algorithm de 256 bits.

Se distinguen tres niveles de respuesta que dan nombre a las columnas de las tablas siguientes. La cobertura observada corresponde a las instancias anotadas dentro del encuadre (reportadas como «Visibles» o GT); el desempeño del detector compara sus predicciones con esas instancias mediante detecciones «Correctas» (TP), predicciones «Falsas» (FP), instancias no recuperadas (FN), precisión, recall, F1 y mAP; y la selección para la voz conserva hasta dos detecciones por fotograma («Correctas (lista)»). Un objeto fuera del encuadre reduce las instancias visibles, pero no es un FN del detector. Sin identificadores entre vistas, no se calcula la cobertura de objetos únicos de un recinto.

4.1.5 Comparación pareada y alcance estadístico

El emparejamiento estadístico compara dos configuraciones dentro de un mismo grupo, en lugar de contrastar fotografías sin relación. Por ejemplo, para cada grupo g (de G01 a G33) se calcula $\Delta g = TP_{g,A} - TP_{g,B}$; la media de esas 33 diferencias describe la ganancia por grupo. Al agrupar lentes, primero se suman dentro del grupo los dos conteos de cada montaje. Se aplica una prueba t de Student pareada con 33 pares y 32 grados de libertad (gl) (NIST/SEMATECH, s. f.). Los intervalos de confianza (IC) del 95 % se obtienen mediante 10.000 remuestreos bootstrap de grupos completos, con semilla 20261013 (Efron, 1979; Field & Welsh, 2007). El remuestreo conserva juntas las seis vistas. Las comparaciones son exploratorias, sin ajuste por multiplicidad, y compartir un mismo recinto entre grupos puede reducir su independencia.

Tabla 4.3. Diseño y unidades de análisis.

Análisis	**Factor manipulado**	**Datos**	**Comparación estadística**
4.2 Montajes	Posición de montaje (3 niveles, lentes agrupadas)	66 fotos por posición	t pareada por grupo; bootstrap de grupos
4.3 Montaje y lente	Posición × lente (6 combinaciones)	33 fotos por combinación	t pareada por grupo; bootstrap de grupos
4.4 Distancia estimada	Distancia objeto-cámara (estimada, 6 bandas)	889 instancias en 198 fotos	Recall por banda con IC bootstrap

Fuente: elaboración propia con datos y especificaciones del proyecto.

4.2 Comparación de posiciones de montaje

El primer análisis agrupa los dos lentes y compara tres montajes. Cada montaje aporta dos fotografías por grupo y 66 fotografías en total. El propósito es describir por separado cuántas instancias quedan dentro de sus encuadres y cuántas de ellas recupera el detector. Los conteos no identifican objetos únicos entre vistas y no aíslan el efecto mecánico del soporte de la orientación usada al capturar.

Tabla 4.4. Resultados agrupados por montaje.

Posición	**Visibles**	**Correctas**	**Falsas**	**Precisión**	**Recall**	**F1**
Arnés de cabeza	293	126	56	69,2%	43,0%	53,1%
Arnés de pecho	277	142	53	72,8%	51,3%	60,2%
Palo de selfie	319	157	60	72,4%	49,2%	58,6%

Fuente: elaboración propia con datos y especificaciones del proyecto.

Cada posición reúne 66 fotografías (33 grupos × 2 lentes). La tabla presenta conteos y métricas del detector; el análisis de la lista de dos objetos se describe al final de esta sección.

El pecho presenta la mayor F1 agrupada (60,2 %); el palo de selfie reúne más instancias visibles (319) y más detecciones correctas (157); la cabeza obtiene 126 correctas de 293 visibles y F1 de 53,1 %. La comparación muestra por qué cobertura y exactitud no son equivalentes: recuperar más instancias en total puede coexistir con una menor proporción recuperada. Las 13 imágenes de cabeza sin objetos objetivo anotados, frente a dos de pecho y una del palo de selfie, describen diferencias de encuadre; no demuestran por sí solas un fallo de clasificación.

La prueba t pareada por grupo (lentes agrupadas) contrasta las posiciones dentro de cada grupo de captura; con gl = 32 el valor crítico a dos colas es t = 2,04 (α = 0,05) (NIST/SEMATECH, s. f.).

Tabla 4.5. Pruebas t pareadas por montaje.

Posición A	**Posición B**	Diferencia media (A−B) objetos/grupo	**t**	**gl**	**p (dos colas)**
Arnés de cabeza	Arnés de pecho	-0,485	-1,49	32	0,1468
Arnés de cabeza	Palo de selfie	-0,939	-3,71	32	0,0008
Arnés de pecho	Palo de selfie	-0,455	-1,63	32	0,1130

Fuente: elaboración propia con datos y especificaciones del proyecto.

Diferencias por grupo del número de objetos correctos entre posiciones, prueba t pareada (gl = 32).

El palo de selfie supera significativamente a la cabeza (t(32) = -3,71, p = 0,0008). Frente al pecho, la diferencia no alcanza el umbral exploratorio de 0,05 (t(32) = -1,63, p = 0,1130); la diferencia entre cabeza y pecho no es significativa (t(32) = -1,49, p = 0,1468). En conjunto: la vista de cabeza ofrece menor cobertura en este conjunto; entre pecho y palo de selfie el mayor número de objetos del palo de selfie convive con un margen relativamente estrecho (0,455 objetos/grupo), lo que anticipa el análisis por configuración de la sección 4.3. La Figura 4.3 muestra la dispersión de los conteos.

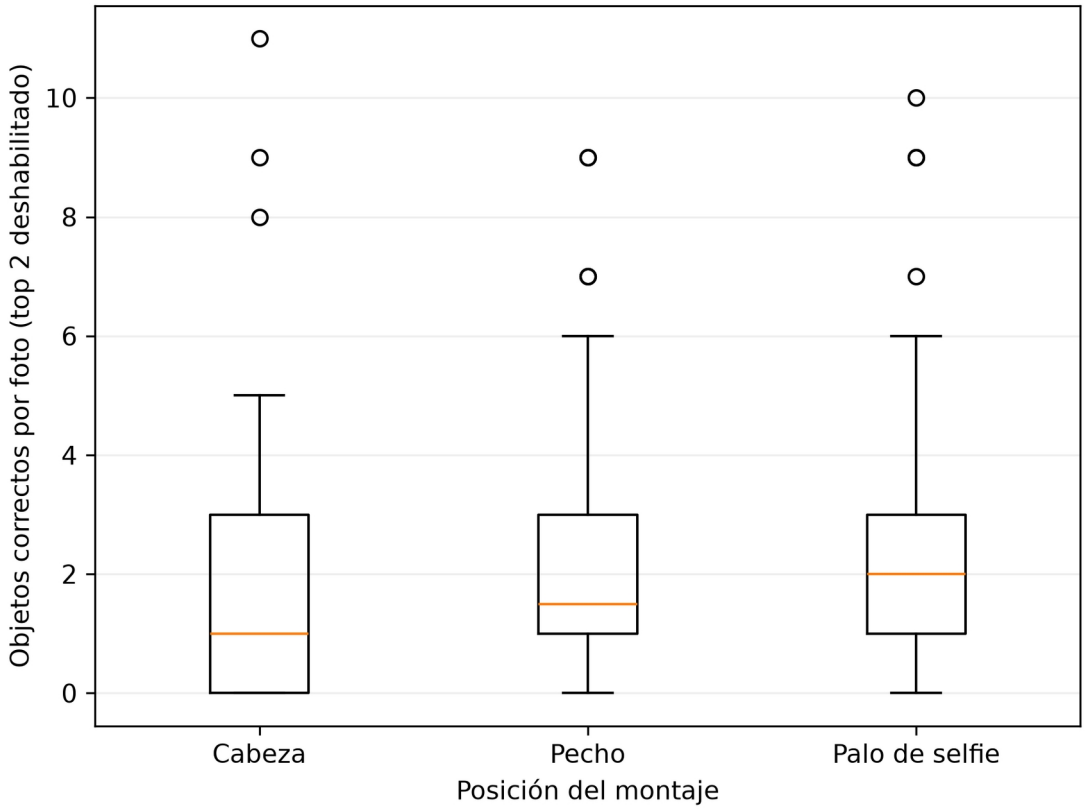


Figura 4.3. Objetos correctos por fotografía y montaje.

Fuente: elaboración propia a partir del diseño y los resultados del proyecto.

Con el límite de dos objetos por fotograma, la lista de anuncios conserva 68 selecciones correctas de la cabeza, 81 del pecho y 89 del palo de selfie, con precisiones de 82,9%, 84,4% y 86,4% respectivamente: la restricción de salida prioriza la calidad de lo que se anuncia sobre la cantidad recuperada.

4.3 Comparación de montaje y lente

El segundo análisis conserva por separado las seis combinaciones de la Tabla 4.2. Cada una aporta 33 fotografías y se compara con las otras dentro de los mismos grupos. Las etiquetas 1x y 0,5x se mantienen como condiciones nominales del registro, sujetas a las discrepancias de metadatos descritas en la sección 4.6.

Tabla 4.6. Resultados por configuración.

Configuración	Visibles	Correctas	Falsas	F1	mAP50 / mAP50-95	Correctas (lista)
Arnés de cabeza 1x	129	56	20	54,6%	53,4% / 33,0%	31
Arnés de cabeza 0,5x	164	70	36	51,9%	50,9% / 33,7%	37
Arnés de pecho 1x	124	65	21	61,9%	60,5% / 42,2%	37
Arnés de pecho 0,5x	153	77	32	58,8%	55,4% / 38,6%	44
Palo de selfie 1x	145	75	29	60,2%	61,9% / 39,0%	40
Palo de selfie 0,5x	174	82	31	57,1%	56,5% / 41,5%	49

Fuente: elaboración propia con datos y especificaciones del proyecto.

Correctas y falsas se contabilizan con confianza 0,30 e IoU ≥ 0,50 sobre las 33 fotografías por configuración; AP se calcula con las predicciones conservadas desde 0,001 y el límite de 15; "Correctas (lista)" aplica el límite de dos objetos por fotograma.

La mejor F1 es del arnés de pecho 1x (61,9%; precisión 75,6%, recall 52,4%, mAP50-95 42,2%) y el máximo de objetos correctos es del palo de selfie 0,5x (82 de 174 instancias visibles). Para explorar la cobertura de detección disponible antes del límite de anuncios, el análisis principal compara el palo de selfie 0,5x contra cada alternativa en los mismos 33 grupos de captura.

La comparación de lentes agrupando montajes tampoco es concluyente: F1 59,0% para 1x frente a 55,9% para 0,5x (diferencia pareada +3,1 puntos de F1, IC 95 % [-2,2; 8,9]). La vista amplia contiene 491 instancias visibles frente a 398 del 1x (+23,4 %) y recupera 229 correctas frente a 196 (+16,8 %): la ganancia de conteo convive con más predicciones falsas por fotografía y menor F1.

Tabla 4.7. Comparación de conteos con el palo de selfie 0,5x.

Alternativa	Grupos + / = / −	Ganancia total de objetos	Ganancia/ grupo	IC 95 % (bootstrap)	p (t pareada)
Arnés de cabeza 1x	19 / 9 / 5	+26	+0,788	[0,333, 1,273]	0,0023
Arnés de cabeza 0,5x	10 / 20 / 3	+12	+0,364	[0,091, 0,667]	0,0260
Arnés de pecho 1x	16 / 10 / 7	+17	+0,515	[0,121, 0,939]	0,0194
Arnés de pecho 0,5x	6 / 24 / 3	+5	+0,152	[-0,182, 0,515]	0,4064
Palo de selfie 1x	11 / 17 / 5	+7	+0,212	[-0,242, 0,667]	0,3702

Fuente: elaboración propia con datos y especificaciones del proyecto.

Grupos + / = / − cuenta en cuántos de los 33 grupos el palo de selfie 0,5x detectó más, igual o menos objetos correctos que la alternativa. El IC proviene de un bootstrap percentil de 10 000 remuestreos pareados por grupos (semilla 20261013, sin corrección por comparaciones múltiples); p proviene de la prueba t pareada de objetos correctos por grupo.

Frente al arnés de pecho 1x la ventaja es de +0,515 objetos por grupo (IC 95 % [0,121; 0,939]; p = 0,019), y frente al pecho 0,5x se reduce a +0,152 con un intervalo que incluye el cero (p = 0,406), de modo que esa ventaja no queda establecida. Se obtiene al menos una detección correcta en 29 de 33 grupos, y el mejor conteo no nulo por grupo en 20 de 33 (empates incluidos).

La Figura 4.4 muestra la distribución de conteos en las seis configuraciones.

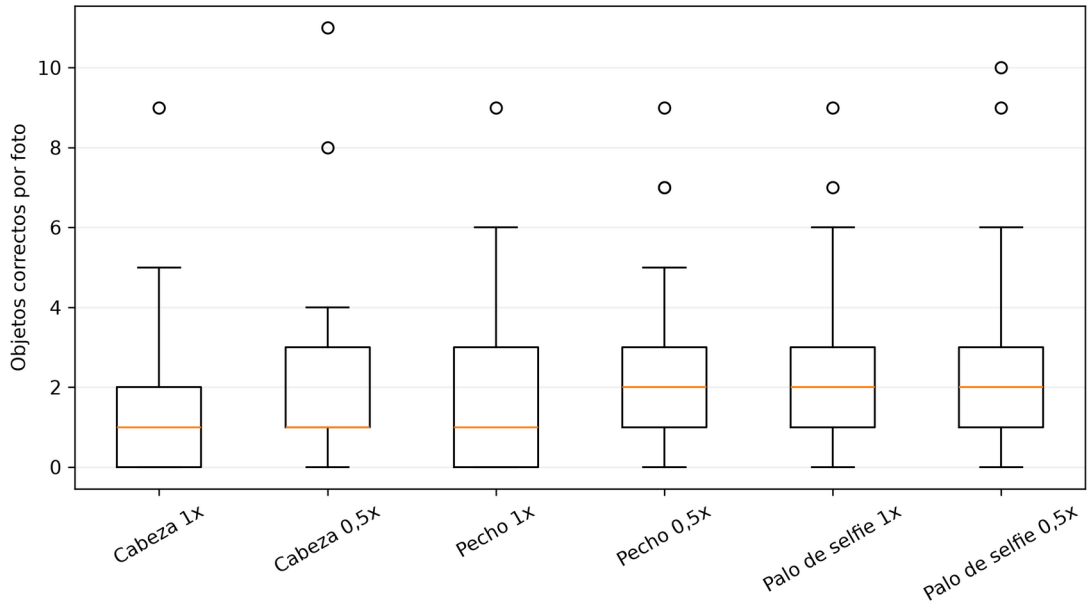


Figura 4.4. Objetos correctos por fotografía en las seis configuraciones.

Fuente: elaboración propia a partir del diseño y los resultados del proyecto.

La elección definitiva depende del objetivo del sistema: para priorizar el número de detecciones correctas antes de limitar la salida de voz, el palo de selfie 0,5x es el candidato provisional; para maximizar el equilibrio precisión/recall por fotograma, el arnés de pecho 1x. En ambos casos los intervalos no descartan todas las alternativas, por lo que la confirmación en escenas nuevas forma parte del plan de validación (sección 4.6).

4.4 Detección por bandas de distancia estimada

Para caracterizar el rango de profundidad útil del sistema se estima la distancia entre el objeto y la cámara para cada instancia anotada de las mismas 198 fotografías, sin capturas adicionales. El modelo estenopeico de la cámara entrega $d \approx f_{px} \cdot H_{cls} / h_{px}$ (Szeliski, 2011), donde f_{px} es la distancia focal en píxeles —calculada del campo FocalLengthIn35mmFilm de los metadatos EXIF (Camera & Imaging Products Association, 2017) (equivalente a 25 mm para el lente 1x y a 14 mm para el 0,5x; se registraron seis discrepancias de grupo de lente y una variación menor de 25 a 26 mm y se adoptó el valor de los metadatos EXIF)—, h_{px} es la altura de la caja anotada en píxeles y H_{cls} es una altura de referencia fija por clase, adoptada como supuesto de este análisis (persona 1,70 m; nevera 1,75 m; silla 0,90 m; sofá 0,85 m; inodoro 0,78 m; mesa de comedor 0,76 m; lavamanos 0,80 m; televisor 0,60 m; cama 0,60 m; computador portátil 0,25 m). Las distancias resultantes son aproximaciones, no mediciones, por lo que el análisis se reporta exclusivamente en bandas.

La tabla siguiente resume el recall del detector por banda de distancia estimada agregando las seis configuraciones. "Recall" se calcula agrupando todas las instancias de la banda; "Media bootstrap" y su IC 95 % se obtienen remuestreando los 33 grupos de captura completos y calculando el recall agrupado en cada remuestra (10 000 remuestreos, semilla 20261013); no son una media de recalls igualmente ponderados por grupo.

Tabla 4.8. Recall por banda de distancia estimada.

Banda de distancia	Instancias	Correctas	Recall	Media bootstrap	IC 95 %	Clases dominantes
0,5-1,0 m	73	55	75,3%	75,7%	[57,0%; 94,1%]	cama 52 %, portátil 18 %, silla 14 %
1,0-2,0 m	160	106	66,2%	66,6%	[54,7%; 78,4%]	silla 41 %, inodoro 12 %, portátil 11 %
2,0-3,0 m	120	75	62,5%	62,3%	[50,0%; 73,4%]	silla 53 %, mesa 23 %, inodoro 7 %
3,0-4,0 m	143	89	62,2%	62,6%	[45,9%; 76,2%]	silla 59 %, televisor 13 %, mesa 10 %
4,0-6,0 m	142	52	36,6%	36,4%	[25,0%; 48,1%]	silla 58 %, televisor 14 %, mesa 12 %
6,0+ m	245	43	17,6%	16,5%	[7,9%; 21,4%]	silla 53 %, lavamanos 15 %, persona 13 %

Fuente: elaboración propia con datos y especificaciones del proyecto.

Detector: confianza 0,30, IoU ≥ 0,50 y coincidencia de clase. Las clases dominantes muestran la proporción de la banda que ocupan las tres clases más frecuentes: la mezcla de clases cambia con la distancia, de modo que profundidad y clase están parcialmente confundidas.

La Figura 4.5 representa el recall por banda de distancia estimada y su intervalo bootstrap.

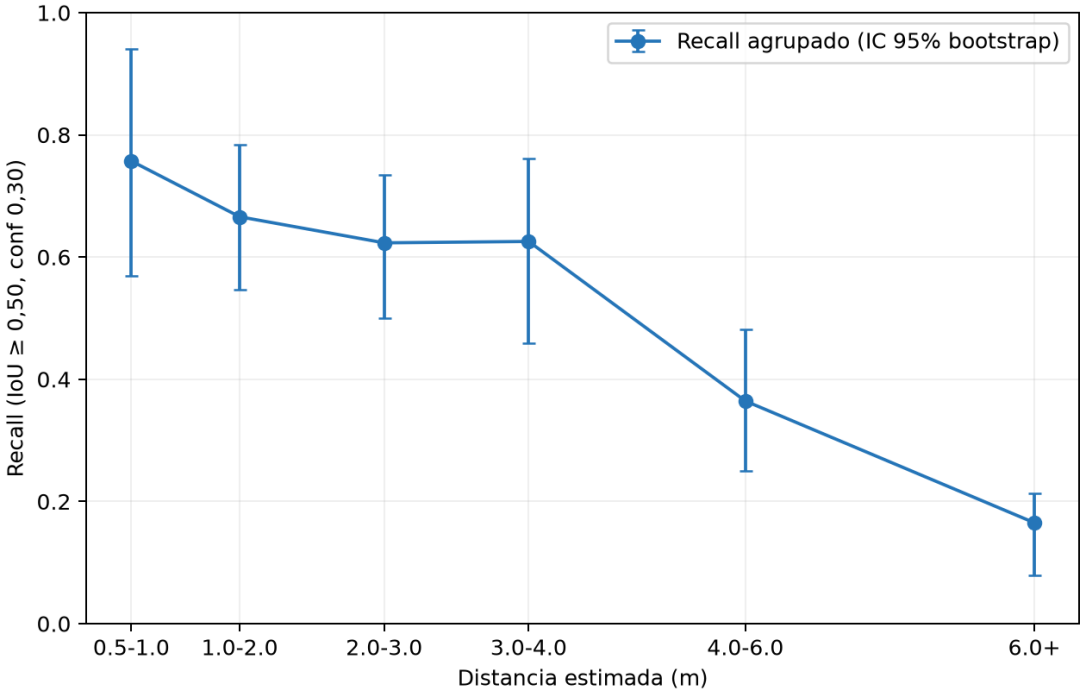


Figura 4.5. Recall por distancia estimada e intervalo bootstrap del 95 %.

Fuente: elaboración propia a partir del diseño y los resultados del proyecto.

El *recall* se mantiene entre 60 % y 75 % hasta aproximadamente 4 m (bandas 0,5-1 m: 75,3%; 1-2 m: 66,2%; 2-3 m: 62,5%; 3-4 m: 62,2%), cae a 36,6% en 4-6 m y colapsa a 17,6% a partir de 6 m. Bajo un criterio operativo de *recall* ≥ 50 % con al menos 30 instancias anotadas por banda, el rango de detección sugerido para el conjunto muestra una transición alrededor de la banda de 4 m. Esta observación no permite fijar un límite operativo ni filtrar objetos por distancia: las distancias no fueron medidas y la aplicación no estima profundidad.

Entre posiciones también hay matices en las bandas lejanas: a más de 6 m, el palo de selfie 1x preserva el mejor recall (26,7% con 45 instancias) frente a 16,3% del arnés de cabeza y 21,2% del de pecho; y el ultra gran angular pierde algo más a distancia (palo de selfie 0,5x: 6,8%). Como en las bandas lejanas dominan sillas, mesas y lavamanos vistos de lejos, ese margen es únicamente exploratorio.

Tres limitaciones condicionan esta conclusión. Primero, la incertidumbre de la altura de referencia por clase no se cuantificó empíricamente, por lo que una instancia puede desplazarse de banda. Segundo, los objetos parcialmente ocultos pierden altura visible en la caja, lo que sobreestima su distancia. Tercero, la mezcla de clases cambia con la distancia (las bandas cercanas concentran camas y computadores; las lejanas, sillas), por lo que el *recall* por banda mezcla profundidad con dificultad por clase. El resultado no caracteriza el sensor ni justifica limitar el asistente a 4 m. Se requiere validación con distancias medidas en escenas nuevas. Los intervalos *bootstrap* describen el muestreo de instancias por grupos y no incorporan el error de estimación de distancia.

4.5 Desempeño por clase y selección para la salida de voz

El desempeño agrupado se complementa con resultados por clase y con la lista de hasta dos detecciones seleccionadas para componer el mensaje. Estas vistas responden a preguntas distintas: qué objetos recupera el detector y qué parte de esa información llega a la selección textual. El límite de salida no cambia el conjunto de instancias anotadas utilizado como denominador del recall.

Tabla 4.9. Resultados por clase en el conjunto completo.

Clase	TP	FP	FN	Precisión	Recall	F1
bed	44	2	12	95,7%	78,6%	86,3%
chair	194	88	244	68,8%	44,3%	53,9%
couch	6	5	5	54,5%	54,5%	54,5%
dining table	28	14	79	66,7%	26,2%	37,6%
laptop	19	11	15	63,3%	55,9%	59,4%
person	39	16	17	70,9%	69,6%	70,3%
refrigerator	24	4	19	85,7%	55,8%	67,6%
sink	18	7	38	72,0%	32,1%	44,4%
toilet	26	2	12	92,9%	68,4%	78,8%
tv	27	20	23	57,4%	54,0%	55,7%

Fuente: elaboración propia con datos y especificaciones del proyecto.

Resultados por clase agrupados sobre las 198 fotografías, con confianza 0,30 e IoU mínimo 0,50. Las diferencias entre clases impiden interpretar el total como utilidad uniforme para todos los objetos.

Los resultados por clase de la Tabla 4.9 describen qué objetos resultan más difíciles; la Figura 4.6 complementa ese desglose comparando precisión, recall y F1 de las seis configuraciones. En cada panel se distinguen el detector y la lista de hasta dos objetos por fotografía. Ambos se evalúan con confianza 0,30, coincidencia de clase e IoU mínimo de 0,50, y el recall conserva como denominador todas las instancias anotadas visibles. Cada configuración reúne 33 fotografías; la escala común de 0 a 100 % permite comparar las magnitudes sin alterar sus definiciones.

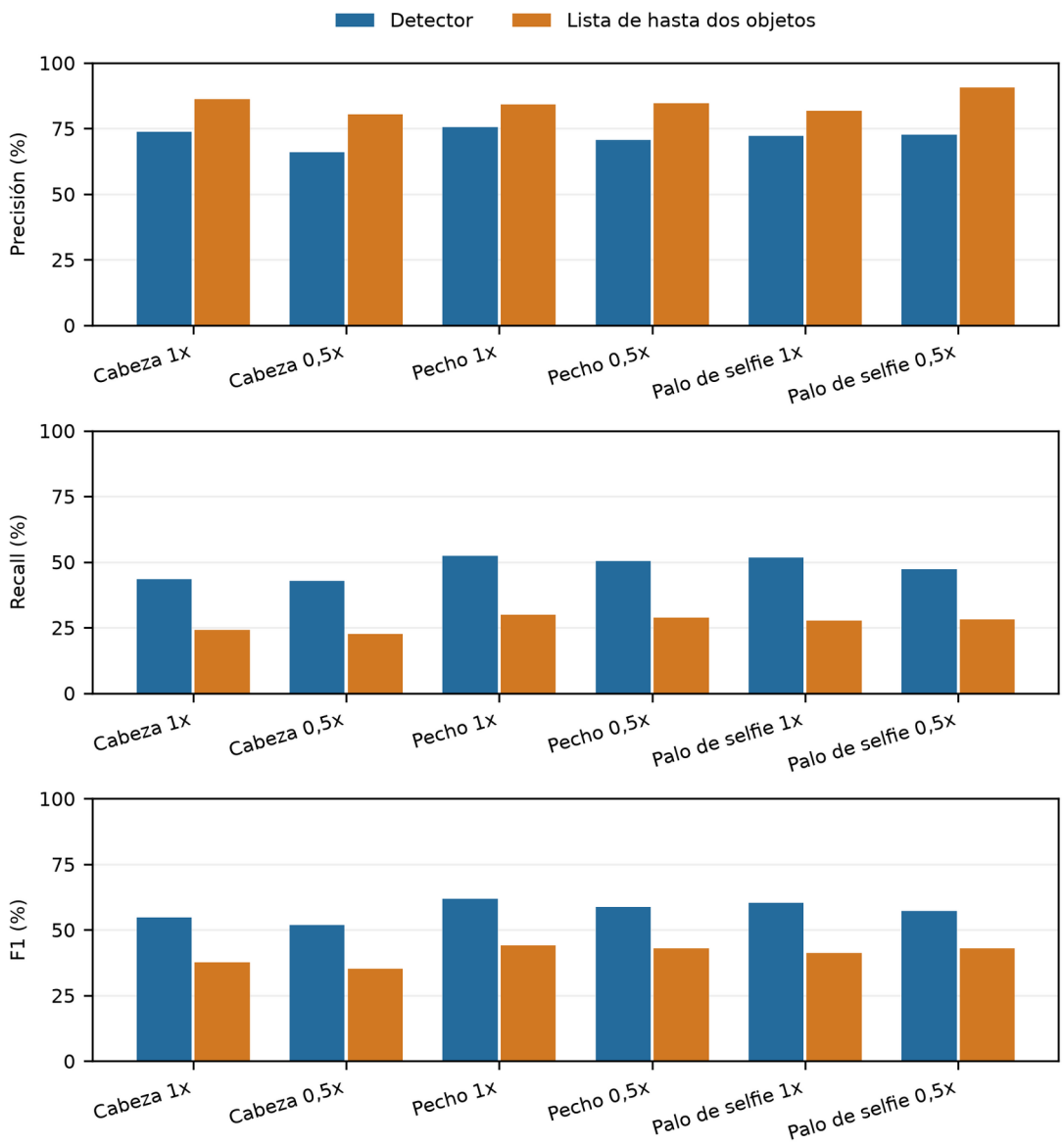


Figura 4.6. Precisión, recall y F1 del detector y de la lista de hasta dos objetos.

Fuente: elaboración propia a partir del diseño y los resultados del proyecto.

La Figura 4.6 permite interpretar el costo de limitar la información anunciada. Para el palo de selfie 0,5x, la precisión pasa de 72,6 % en el detector a 90,7 % en la lista, mientras el recall baja de 47,1 % a 28,2 %: se conservan 49 selecciones correctas frente a las 82 detecciones correctas previas al límite. Pecho 1x mantiene la mayor F1 del detector (61,9 %), aunque recupera menos objetos correctos en total. La lista prioriza las puntuaciones más altas y reduce la cantidad recuperada; su mayor precisión no demuestra por sí sola mejor comprensión o utilidad del mensaje. Por ello, la elección combina esta figura con los

conteos de la sección 4.3 y conserva la selección del palo de selfie 0,5x como provisional cuando se prioriza cantidad.

La Tabla 4.6 aporta una dimensión adicional: mAP50-95 promedia umbrales de IoU más exigentes que mAP50. En pecho 1x se obtienen 60,5 % y 42,2 %, respectivamente; la diferencia describe la sensibilidad de la evaluación al ajuste de las cajas y no aísla su causa. La Figura 4.6 utiliza un IoU fijo de 0,50, por lo que complementa ese resultado sin sustituirlo. El efecto de las imprecisiones de las cajas sobre la zona espacial anunciada requiere una evaluación específica.

4.6 Consideraciones de validez

4.6.1 Validez interna

El emparejamiento por grupo mantiene una referencia de captura común, pero no hace idénticos los encuadres. Los grupos se reconstruyeron según el orden ascendente de los archivos, con estado provisional en capture_groups.csv, por lo que dicho registro no constituye una verificación independiente de los recintos. Las tomas son sucesivas y pueden cambiar la pose o la presencia de personas. Las cajas fueron corregidas por el autor, sin segunda auditoría humana independiente. Además, el palo de selfie se orientó hacia los objetos: se compara soporte junto con estrategia de encuadre. Elegir una configuración en este mismo conjunto favorece al ganador observado y exige confirmación en escenas nuevas.

4.6.2 Correspondencia entre métricas y uso previsto

Las métricas cuentan instancias visibles en cada fotografía, no objetos únicos del mundo: un mismo escritorio fotografiado desde tres posiciones genera tres instancias y tres oportunidades de acierto, y un objeto fuera de encuadre no penaliza al detector. La lista de dos objetos modela la salida de la aplicación, pero no mide la calidad del habla ni la utilidad para el usuario con discapacidad visual. Las sillas representan 49,3% de las instancias, de modo que las métricas agrupadas heredan ese desbalance; las métricas por clase acompañan el reporte y deben consultarse antes de sacar conclusiones de utilidad por objeto.

4.6.3 Alcance de la generalización

Las fotografías provienen de un iPhone 12 Pro Max y la inferencia corrió en un perfil de emulador Pixel 8 (Android 17, API 37, x86_64): las latencias reportadas (mediana 246,3 ms por detección y 104,0 ms por decodificación + preprocesamiento) son mediciones de depuración sobre emulador y no extrapolan al teléfono de destino. Las ópticas del lente 0,5x del iPhone no se trasferirán directamente al teléfono Android del prototipo. El muestreo concentra espacios domésticos e institucionales y contiene también vistas de terrazas; se restringe a diez clases COCO. La generalización a otros exteriores, iluminación nocturna u otras clases queda fuera del alcance de este capítulo.

4.6.4 Incertidumbre de las distancias

Las distancias se estiman a partir de metadatos y alturas de referencia por clase, sin medición física ni calibración de cada objeto. La oclusión puede reducir la altura visible y sobreestimar la distancia; la mezcla de clases cambia entre bandas. La caída de recall alrededor de las bandas de 4 m describe este archivo y no establece un alcance operativo de la aplicación.

4.6.5 Alcance de las comparaciones estadísticas

La prueba t pareada usa 33 grupos por comparación (gl = 32) y el bootstrap 10 000 remuestreos por grupos de captura (semilla 20261013); ambos se reportan como análisis exploratorio. Los intervalos no se ajustan ni por la correlación entre grupos vecinos ni por las comparaciones pareadas múltiples entre configuraciones; los p-valores y los intervalos describen este conjunto de datos y no recomiendan por sí solos una configuración de despliegue. La agenda de trabajo posterior comprende una segunda revisión independiente de anotaciones, la validación en escenas nuevas con el teléfono físico, un protocolo con distancias físicas medidas y pruebas de campo con usuarios finales.

La auditoría EXIF identificó seis discrepancias entre el grupo de lente nominal y el campo focal registrado, además de una variación menor de 25 a 26 mm. Se conservan las etiquetas originales del protocolo para mantener la trazabilidad; las comparaciones de lente son provisionales hasta revisar esas capturas. Las bandas publicadas reúnen 883 de las 889 instancias: seis estimaciones inferiores a 0,5 m quedan fuera del gráfico. La

columna Media *bootstrap* es la media de los recalls agrupados calculados en las remuestras, no una media de recalls igualmente ponderados por grupo.

4.7 Distribución del tiempo de procesamiento

Se analizaron los tiempos guardados de las 198 fotografías, sin repetir inferencia. La ejecución retuvo candidatos desde puntuación 0,001; no corresponde a una nueva medición con corte de 0,30. La Tabla 4.10 resume mediana y percentiles: P25, P75 y P95 son los valores bajo los cuales se sitúa aproximadamente el 25 %, 75 % y 95 % de los tiempos. Se calculan mediante interpolación lineal en la posición (n − 1)q, donde n es el número de observaciones y q el cuantil entre 0 y 1 (NumPy developers, s. f.).

Tabla 4.10. Distribución de tiempos de reproducción en emulador en milisegundos.

Etapa	P25	Mediana	P75	P95
Decodificación y preprocesamiento	100,5	104,0	110,1	125,9
Método de detección	238,4	246,3	258,9	307,5
Suma por imagen	340,9	352,4	367,4	419,9

Fuente: elaboración propia con datos y especificaciones del proyecto.

La suma se calculó para cada imagen antes de resumirla: su mediana es 352,4 ms y su P95 es 419,9 ms. No se sumaron las medianas de etapas. El intervalo instrumentado de detección incluye la llamada y preparación inmediata de su medición; no es un perfil de operadores del modelo. Estos tiempos corresponden a archivos del estándar Joint Photographic Experts Group (JPEG) reproducidos en un emulador Pixel 8 con compilación de depuración. Excluyen adquisición de cámara, interfaz, localización, generación de voz y reproducción del mensaje. Por tanto, no representan el tiempo desde pulsar Capturar hasta oír la respuesta, ni el rendimiento de un Pixel 8 físico.

4.8 Brillo digital y oclusión observada

La propuesta contempla iluminación y oclusión (Cerda Leon, s. f.), pero la adquisición no documenta un protocolo de niveles controlados de iluminancia ni porcentajes físicos de superficie oculta. El análisis utiliza las condiciones presentes en las fotografías. El brillo medio se calcula sobre el archivo original, mediante conversión de RGB a escala de grises de ocho bits con Pillow (Pillow contributors, s. f.), y se divide en tres grupos por los cuantiles

1/3 y 2/3. Es una comparación de brillo digital, afectada también por exposición y composición; no una medición en lux ni una prueba de tolerancia a la oscuridad.

Tabla 4.11. Detección agrupada por brillo digital de las fotografías.

Grupo de brillo	Fotos	GT	TP	FP	Recall
Menor brillo	66	326	136	85	41,7 %
Brillo intermedio	66	191	118	26	61,8 %
Mayor brillo	66	372	171	58	46,0 %

Fuente: elaboración propia con datos y especificaciones del proyecto.

Cada grupo reúne 66 fotografías. Los intervalos observados de brillo medio son menor brillo: 88,4 a 124,7; brillo intermedio: 124,8 a 136,8; mayor brillo: 136,8 a 157,1 en una escala de 0 a 255. La Tabla 4.11 utiliza detecciones con confianza 0,30 e IoU mínimo 0,50 y calcula recall agrupando instancias, sin promediar recalls por fotografía. Se conservan las imágenes vacías en el conteo de fotos; no aportan instancias al denominador de recall.

El grupo intermedio presenta mayor *recall* observado, pero no se obtiene una relación monótona con el brillo. La mezcla de clases también cambia: las sillas representan 44,8 %, 41,4 %, 57,3 % de las instancias de los tres grupos, respectivamente. El montaje, lente, fondo y densidad de objetos varían junto con el brillo. La comparación es descriptiva, no una prueba causal de iluminación ni un umbral operativo; no permite afirmar tolerancia a oscuridad o condiciones de lux específicas. El archivo complementario registra los conteos por clase y configuración para hacer visible esta confusión.

4.8.1 Análisis observacional de oclusión

El etiquetado suplementario se apoyó en inteligencia artificial (IA). Se inspeccionaron las 198 fotografías y se asignó una etiqueta suplementaria a cada una de las 889 instancias COCO, sin modificar sus cajas ni volver a ejecutar el detector. Dos lotes disjuntos fueron revisados visualmente con GPT-6 Astra y razonamiento High, sin mostrar predicciones durante el etiquetado. Como definición operativa propia para este análisis, el protocolo distingue ausencia de obstrucción externa, oclusión parcial y fuerte; los casos ambiguos se conservan como inciertos. El truncamiento por borde se registra aparte, y el tamaño pequeño o la perspectiva no se consideran por sí solos oclusión. La severidad es cualitativa y no corresponde a porcentajes medidos de superficie oculta.

El autor revisó 215 de las 889 etiquetas mediante la interfaz local; se conservaron las otras 674 etiquetas asistidas por IA sin revisión individual adicional. Los dos lotes iniciales no constituyen doble revisión independiente. Se registra motivo, confianza e identificador de anotación, y se proporciona una interfaz local para corregir los casos. El análisis utiliza el emparejamiento del *benchmark* con confianza 0,30, IoU mínimo 0,50 y coincidencia de clase. Se calcula *recall* por nivel; los falsos positivos sin objeto de referencia no reciben una etiqueta de oclusión.

Tabla 4.12. Recall por nivel de oclusión con etiquetas asistidas por IA.

Oclusión	GT	TP	FN	Recall	IC 95 %
Sin oclusión	274	186	88	67,9 %	[60,0 %; 75,5 %]
Parcial	408	217	191	53,2 %	[46,6 %; 63,3 %]
Fuerte	168	18	150	10,7 %	[5,4 %; 21,9 %]
Incierta	39	4	35	10,3 %	[0,0 %; 33,3 %]

Fuente: elaboración propia con datos y especificaciones del proyecto.

Los intervalos percentiles del 95 % se obtienen con 10 000 remuestreos de los 33 grupos completos (semilla 20261013). Los casos inciertos se muestran por separado y no se mezclan con los tres niveles de severidad. Los intervalos reflejan variación entre grupos, pero no incertidumbre del etiquetado ni posibles errores de la verdad de campo. El análisis incluye únicamente instancias ya anotadas, por lo que no cubre objetos completamente ocultos y ausentes de la anotación.

El *recall* observado es 67,9 % sin oclusión, 53,2 % con oclusión parcial y 10,7 % con oclusión fuerte. Estos valores describen asociaciones del conjunto; no aíslan el efecto causal de la oclusión. Se conservan desgloses por clase y tamaño aparente conjunto, configuración y truncamiento, además de análisis de sensibilidad con etiquetas de alta confianza y objetos no truncados. El tamaño aparente se define por área de caja dividida entre área de imagen: pequeño < 0,01, mediano de 0,01 a < 0,10 y grande ≥ 0,10; no representa el área completa de la parte oculta.

La composición limita la interpretación del gradiente: 130 de las 168 instancias con oclusión fuerte son sillas y 137 tienen caja pequeña, frente a 51 de 274 sin oclusión. Dentro de sillas de tamaño mediano, el *recall* es 75,0 % (54/72), 62,0 % (62/100), 27,3 % (6/22) para ausencia de oclusión, parcial y fuerte, respectivamente; aún varían escena, pose y tamaño dentro de la banda. En camas se obtiene 66,7 % sin oclusión y 89,7 % con oclusión parcial, por lo que el gradiente agrupado no debe asumirse universal. Al excluir objetos truncados,

los valores agrupados son 77,6 %, 49,7 %, 10,7 %; con etiquetas de alta confianza son 69,9 %, 54,8 %, 11,1 %, en el mismo orden de severidad. Estas comprobaciones describen sensibilidad, pero no eliminan la confusión ni sustituyen la revisión de etiquetas.

La inspección señaló posibles inconsistencias de clase, cajas que abarcan más de una instancia y duplicados; se registraron como incidencias para revisión sin corregir automáticamente la referencia original. Las celdas con menos de 15 instancias se identifican como escasas. La revisión de etiquetas no modifica las cajas ni clases originales. Antes de formular conclusiones definitivas, deben resolverse las etiquetas ambiguas y las incidencias GT, y recalcularse los resultados si cambia la referencia. Las tablas detalladas, los hashes y el protocolo se conservan en Build/Testing/occlusion/.

4.9 Evaluación de la retroalimentación auditiva

Para responder al tercer objetivo en entornos interiores controlados, la claridad de la retroalimentación auditiva se evaluó técnicamente en tres dimensiones complementarias: la claridad informativa del contenido seleccionado para el anuncio, la claridad sintáctica y espacial de las oraciones construidas, y la ejecución de la síntesis neuronal local fuera de línea.

En la dimensión informativa (sección 4.5 y Figura 4.6), el filtrado por confianza (≥ 0,30) y la restricción por defecto a un máximo de dos objetos prioritarios por captura evitaron anuncios extensos o saturados de falsos positivos: la precisión de los elementos comunicados por voz ascendió de 72,6 % a 90,7 % en el palo de selfie 0,5x y de 75,6 % a 88,1 % en el arnés de pecho 1x (alcanzando 86,4 % y 84,4 % al agrupar lentes). De este modo, casi nueve de cada diez objetos incluidos en la descripción verbal corresponden a instancias verdaderas correctamente localizadas.

En la dimensión sintáctica y espacial, se ejecutaron 13 pruebas unitarias automatizadas en la JVM con un 100 % de aprobación: seis pruebas sobre LocalizationModule verificaron la asignación determinista del centro normalizado (u, v) a las seis regiones del encuadre, y siete pruebas sobre SpeechFormatter comprobaron la construcción de frases completas y sin ambigüedad gramatical en español e inglés (incluyendo concordancia de artículos, conectores para uno, dos o tres objetos y notificación de escena sin detecciones).

Finalmente, en la dimensión de síntesis y reproducción local, el módulo de voz superó el 100 % de las pruebas de configuración y ejecución: siete pruebas unitarias de TtsVoiceCatalog validaron los parámetros e identificadores bilingües, y cinco pruebas de instrumentación en Android ejecutadas con modo avión confirmaron la integridad de los 362 activos fonéticos (espeak-ng-data), la inicialización de los modelos VITS/Piper (es_MX-claude-high-int8 y en_US-amy-low-int8) con sherpa-onnx, la generación íntegra de la señal de voz en el dispositivo y su emisión mediante AudioTrack con liberación automática de la captura. En conjunto, estos resultados acreditan el cumplimiento de los tres criterios técnicos del tercer objetivo.

El anuncio describe la escena mediante una oración y no incluye un saludo inicial. Por ejemplo, si se seleccionan una silla en la región inferior izquierda y un televisor en la superior derecha, SpeechFormatter construye: «Hay una silla abajo a la izquierda de donde estás apuntando y un televisor arriba a la derecha de donde estás apuntando». Este ejemplo ilustra la composición prevista por el código, no una nueva observación experimental. Las regiones se refieren al encuadre de la cámara y el mensaje no expresa una distancia física. Cuando no hay detecciones seleccionadas, la implementación muestra el estado «sin detecciones» y libera la captura; no genera una descripción de objetos. Las pruebas técnicas de formato y síntesis no miden por sí solas la comprensión del mensaje por parte de un oyente.

5. Conclusiones

En este trabajo se desarrolló un prototipo móvil para identificar objetos de entornos interiores y comunicar su ubicación relativa mediante voz, como apoyo a la percepción del entorno de personas con discapacidad visual. El desarrollo se organizó alrededor de la selección del detector, la integración del sistema Android y su evaluación técnica. Los hallazgos de estas etapas permiten valorar los objetivos y precisar el alcance de la solución.

El primer objetivo se atendió con la selección de EfficientDet-Lite2 en CPU. La comparación de seis configuraciones mostró un compromiso entre mAP50, recall, latencia y memoria: el modelo obtuvo 0,648, 0,677, 124,9 ms y 65,8 MB, respectivamente. La latencia fue decisiva por su contribución a la espera previa al mensaje de voz; frente a YOLOv8s Float32 se aceptó una mAP50-95 inferior a cambio de reducir el tiempo de inferencia a aproximadamente una quinta parte. La selección corresponde al conjunto y al teléfono evaluados y no establece una superioridad universal. La comparación mostró que la arquitectura debe elegirse según el uso previsto: YOLOv8s ajustó mejor las cajas en umbrales exigentes, SSD MobileNetV1 procesó más rápido pero recuperó menos objetos y EfficientDet-Lite2 ofreció el compromiso seleccionado entre recuperación, tiempo y memoria. Reducir la espera antes del anuncio auditivo justificó priorizar este último equilibrio.

En cuanto al segundo objetivo, se desarrolló un sistema integrado de asistencia visual en Android que conecta captura bajo demanda, preprocesamiento, detección, selección configurable de objetos, localización relativa en seis zonas y descripción auditiva local en español e inglés. La arquitectura modular incorpora CameraX, EfficientDet-Lite2, geometría letterbox y síntesis de voz con sherpa-onnx. La selección del detector se sustentó en mAP50 de 0,648, recall de 0,677 y latencia aislada de 124,9 ms en Redmi

Note 13. Las 34 pruebas JVM y las pruebas de instrumentación en Android respaldan la verificación completa de los módulos de visión, localización y síntesis de voz local.

A partir de esta integración, para el tercer objetivo la evaluación de captura completó 198 fotografías, 889 anotaciones revisadas y 33 grupos pareados (G01–G33, en 30 recintos físicos). El palo de selfie 0,5x produjo 82 detecciones correctas y recuperó al menos un objeto en 29 grupos; el arnés de pecho 1x obtuvo la mayor F1, de 61,9 %. Bajo la orientación hacia los objetos utilizada en estas capturas, la elección por número de detecciones correctas favorece provisionalmente el palo de selfie 0,5x, mientras la elección por equilibrio precisión-recall favorece pecho 1x. Las comparaciones no establecen un ganador universal y deben confirmarse en escenas nuevas. La diferencia entre cobertura visual y desempeño del detector es decisiva para interpretar esa elección: el palo de selfie 0,5x incluyó 174 instancias y recuperó 82, mientras pecho 1x incluyó 124 y recuperó 65. El primero aportó más detecciones en términos absolutos; el segundo recuperó una mayor proporción de lo visible y obtuvo mayor F1. Un objeto fuera del encuadre reduce la información adquirida, pero no constituye un falso negativo del detector.

Los conteos por grupo mostraron una ventaja del palo de selfie 0,5x frente a pecho 1x de 17 detecciones (0,515 por grupo; IC bootstrap del 95 % de 0,121 a 0,939). Frente a pecho 0,5x, el intervalo incluyó cero. Al agrupar lentes, la diferencia palo de selfie-pecho no fue significativa en la prueba t corregida ($p = 0,113$). En la evaluación de la retroalimentación auditiva, limitar el anuncio a hasta dos objetos por captura elevó la precisión de las selecciones enviadas a voz hasta 90,7 % en el palo de selfie 0,5x (88,1 % en pecho 1x), lo que junto con las 14 pruebas unitarias de formato y catálogo bilingüe y las cinco pruebas instrumentadas de síntesis sherpa-onnx en modo avión respalda la selección del contenido, la construcción de las frases y la ejecución local de la voz, sin medir su inteligibilidad con participantes.

La exploración de profundidad sugiere menor recall en las bandas geométricamente estimadas más lejanas, pero no establece un alcance físico de 4 m. El tamaño de referencia de cada clase, la oclusión y la composición de objetos confunden la relación con la distancia. Respecto del objetivo general, el aporte demostrado es un sistema móvil funcional, autónomo y reproducible que integra detección, localización espacial en seis

zonas y descripción auditiva local, evaluado en exactitud, latencia y claridad de su retroalimentación.

Respecto a las fases propuestas, se completaron la revisión conceptual, la selección experimental del detector, el desarrollo integrado del prototipo y su evaluación técnica en escenarios interiores. El análisis de tiempos registró una mediana de 352,4 ms y un P95 de 419,9 ms para decodificación, preprocesamiento y detección en emulador, complementado con los análisis de distancia estimada, brillo digital y oclusión observada.

Las principales dificultades fueron la variación de encuadre entre montajes, las discrepancias entre algunas etiquetas de lente y los metadatos, y la separación entre las mediciones históricas del teléfono y la reproducción en emulador. A ello se añaden la distancia estimada sin calibración, las etiquetas de oclusión parcialmente asistidas por IA y la ausencia de una evaluación de comprensión con usuarios. Estas limitaciones impiden atribuir los cambios únicamente al soporte, fijar un alcance físico o extrapolar los tiempos y la utilidad a cualquier dispositivo o entorno.

Se recomienda utilizar conjuntamente conteos, cobertura, precisión, *recall* y métricas por clase; revisar las discrepancias EXIF antes de fijar conclusiones sobre lentes; mantener separados los resultados históricos de Redmi y los de emulador; conservar imágenes, etiquetas, predicciones, código y versiones; y reproducir la selección en el teléfono de destino. La salida de voz debe regularse según pruebas de comprensión y carga informativa, no solo según el número máximo de objetos recuperados.

El trabajo posterior comprende una segunda revisión independiente de anotaciones; identificación de objetos compartidos entre vistas; validación del palo de selfie 0,5x en escenas nuevas y con el teléfono físico; un protocolo de distancias medidas y calibración óptica; y evaluación de inteligibilidad, tiempos de respuesta y uso con personas con discapacidad visual. La síntesis local ya está integrada; su comparación con otros motores y la incorporación de audio espacial, vibración o exploración continua requieren experimentos propios.

Como aprendizaje metodológico, conservar anotaciones, predicciones, temporizaciones y versiones permitió ampliar el informe sin alterar la ejecución original. El etiquetado posterior de oclusión amplía el análisis observacional, mientras que las mediciones en lux y las

pruebas de campo con usuarios finales se proyectan como continuación. Para próximos proyectos del programa, conviene definir desde el inicio los criterios de aceptación y los instrumentos de medición, separar pruebas técnicas de evaluación con usuarios y reservar un conjunto de escenas independiente para confirmar la configuración seleccionada.

Para el propósito de asistencia planteado, el prototipo demuestra la viabilidad técnica de transformar una captura solicitada por el usuario en información verbal sobre objetos y regiones del encuadre, sin conexión a internet. Esa función puede servir como apoyo puntual para explorar lo que hay frente a la cámara. Su utilidad práctica para personas con discapacidad visual debe confirmarse mediante pruebas de comprensión y uso en condiciones reales; los resultados actuales no demuestran una mejora de autonomía ni validan una ayuda de navegación segura.

Anexo A. Repositorio y reproducción de los experimentos

El proyecto se organiza en dos repositorios públicos de GitHub: indoor-detect conserva los conjuntos de datos, los resultados y el informe final, mientras que AssistiveVision aloja el código fuente de la aplicación Android. Para evitar duplicar instrucciones operativas en este documento, todos los requisitos de entorno, comandos de terminal y pasos de reproducción se consultan directamente en el archivo README.md de cada repositorio.

El repositorio https://github.com/diomedescerda/indoor-detect reúne las 198 fotografías, las 889 anotaciones (labels.json), los 33 grupos G01–G33 (capture_groups.csv) y las tablas y figuras del estudio. Su README.md explica cómo navegar los datos y reproducir los análisis con NumPy (NumPy developers, s. f.), pycocotools en la evaluación COCO (COCO dataset contributors, s. f.) y Matplotlib en las figuras (Hunter, 2007).

El repositorio de la aplicación móvil se encuentra en https://github.com/diomedescerda/AssistiveVision (rama principal main, revisión evaluada 85c852ddcb543bce1bd69ad0db307f895d597881) (Cerda Leon, 2026). Su README.md documenta la arquitectura del proyecto y enlaza las instrucciones para compilar el paquete instalable (APK), ejecutar las pruebas unitarias e instrumentadas y utilizar la aplicación.

Para instalar, abrir Descargar APK desde un dispositivo Android compatible y ejecutar el instalador. La sección Versiones y comprobantes de GitHub Releases publica el APK firmado junto con sus sumas SHA-256, el manifiesto de compilación y las licencias correspondientes.

Referencias

Camera & Imaging Products Association. (2017). *Exif 2.31 metadata for XMP* (Draft CIPA DC-X010-2017). Camera & Imaging Products Association. https://www.cipa.jp/std/documents/e/DC-X010-2017.pdf

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. En A. Vedaldi, H. Bischof, T. Brox, & J.-M. Frahm (Eds.), *Computer Vision – ECCV 2020* (pp. 213-229). Springer. https://doi.org/10.1007/978-3-030-58452-8_13

Cerda Leon, D. F. (s. f.). *Sistema de identificación y localización espacial de objetos basado en visión por computador para asistencia a personas con discapacidad visual* [Propuesta de pasantía en investigación]. Universidad del Magdalena.

Cerda Leon, D. F. (2026). *AssistiveVision* (Versión 85c852ddcb543bce1bd69ad0db307f895d597881) [Software]. GitHub. https://github.com/diomedescerda/AssistiveVision/tree/85c852ddcb543bce1bd69ad0db307f895d597881

COCO dataset contributors. (s. f.). *COCO API: PythonAPI/pycocotools/cocoeval.py* [Software]. GitHub. Recuperado 5 de noviembre de 2026, de https://github.com/cocodataset/cocoapi/blob/master/PythonAPI/pycocotools/cocoeval.py

Dai, J. (2020). Real-time and accurate object detection on edge device with TensorFlow Lite. *Journal of Physics: Conference Series*, *1651*(1), 012114. https://doi.org/10.1088/1742-6596/1651/1/012114

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, *7*(1), 1-26. https://doi.org/10.1214/aos/1176344552

Field, C. A., & Welsh, A. H. (2007). Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, *69*(3), 369-390. https://doi.org/10.1111/j.1467-9868.2007.00593.x

Figura, J., Kuźmiczuk, R., & Badurowicz, M. (2025). Analysis of object recognition systems using augmented reality glasses. *Journal of Computer Sciences Institute*, *34*, 8-13. https://doi.org/10.35784/jcsi.6469

Google. (s. f.-a). *Android Debug Bridge (adb)*. Android Developers. Recuperado 5 de noviembre de 2026, de https://developer.android.com/tools/adb

Google. (s. f.-b). *AudioTrack*. Android Developers. Recuperado 5 de noviembre de 2026, de https://developer.android.com/reference/android/media/AudioTrack

Google. (s. f.-c). *CameraX*. Android Developers. Recuperado 5 de noviembre de 2026, de https://developer.android.com/media/camera/camerax

Google. (s. f.-d). *DataStore*. Android Developers. Recuperado 5 de noviembre de 2026, de https://developer.android.com/topic/libraries/architecture/datastore

Google. (s. f.-e). *GPU acceleration delegate with Interpreter API*. Google AI Edge. Recuperado 5 de noviembre de 2026, de https://developers.google.com/edge/litert/android/gpu

Google. (s. f.-f). *LiteRT for Android*. Google AI Edge. Recuperado 5 de noviembre de 2026, de https://developers.google.com/edge/litert/android

Google. (s. f.-g). *Object detection with TensorFlow Lite Model Maker*. Google AI Edge. Recuperado 5 de noviembre de 2026, de https://developers.google.com/edge/litert/libraries/modify/object_detection

Guravaiah, K., Bhavadeesh, Y. S., Shwejan, P., Vardhan, A. H., & Lavanya, S. (2023). Third Eye: Object recognition and speech generation for visually impaired. *Procedia Computer Science, International Conference on Machine Learning and Data Engineering*, *218*, 1144-1155. https://doi.org/10.1016/j.procs.2023.01.093

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). *MobileNets: Efficient convolutional neural networks for mobile vision applications* (arXiv:1704.04861). arXiv. https://doi.org/10.48550/arXiv.1704.04861

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

k2-fsa. (s. f.). *Text-to-speech*. Sherpa-Onnx. Recuperado 5 de noviembre de 2026, de https://k2-fsa.github.io/sherpa/onnx/tts/index.html

Khronos Group. (2013, julio 21). *OpenCL: The open standard for parallel programming of heterogeneous systems*. Khronos Group. https://www.khronos.org/opencl/

Kim, J., Kong, J., & Son, J. (2021). *Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech* (Preprint 2106.06103). arXiv. https://doi.org/10.48550/arXiv.2106.06103

Kuznetsova, A., Rom, H., Alldrin, N., Uijlings, J., Krasin, I., Pont-Tuset, J., Kamali, S., Popov, S., Malloci, M., Kolesnikov, A., Duerig, T., & Ferrari, V. (2020). The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale. *International Journal of Computer Vision*, *128*(7), 1956-1981. https://doi.org/10.1007/s11263-020-01316-z

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. En D. Fleet, T. Pajdla, B. Schiele, & T. Tuytelaars (Eds.), *Computer Vision – ECCV 2014* (pp. 740-755). Springer. https://doi.org/10.1007/978-3-319-10602-1_48

Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., & Berg, A. C. (2016). SSD: Single shot MultiBox detector. En B. Leibe, J. Matas, N. Sebe, & M. Welling (Eds.), *Computer Vision – ECCV 2016* (pp. 21-37). Springer International Publishing. https://doi.org/10.1007/978-3-319-46448-0_2

Magdy, A., Moustafa, M. S., Ebied, H. M., & Tolba, M. F. (2025). Lightweight faster R-CNN for object detection in optical remote sensing images. *Scientific Reports*, *15*(1), 16163. https://doi.org/10.1038/s41598-025-99242-y

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/

Meenakshi, R., Ponnusamy, R., Alghamdi, S., Khalaf, O. I., & Alotaibi, Y. (2022). Development of mobile app to support the mobility of visually impaired people. *Computers, Materials and Continua*, *73*(2), 3473-3495. https://doi.org/10.32604/cmc.2022.028540

More, S. S., Patil, N., Lobo, V. B., Shet, N., Goswami, D., & Rane, P. (2025). Empowering the visually impaired: YOLOv8-based object detection in Android applications. *Procedia Computer Science, 4th International Conference on Evolutionary Computing and Mobile Sustainable Networks*, *252*, 457-469. https://doi.org/10.1016/j.procs.2025.01.005

NIST/SEMATECH. (s. f.). *Analysis of paired observations*. NIST/SEMATECH e-Handbook of Statistical Methods. Recuperado 5 de noviembre de 2026, de https://www.itl.nist.gov/div898/handbook/prc/section3/prc311.htm

NumPy developers. (s. f.). *numpy.quantile*. NumPy Documentation. Recuperado 5 de noviembre de 2026, de https://numpy.org/doc/stable/reference/generated/numpy.quantile.html

ONNX Runtime. (s. f.). *ONNX Runtime documentation*. ONNX Runtime. Recuperado 5 de noviembre de 2026, de https://onnxruntime.ai/docs/

Pawar, D. (2025). *Deep learning-driven assistive technology for blind users: Real-time object detection and speech feedback on Android*. https://doi.org/10.13140/RG.2.2.28125.12005

Pillow contributors. (s. f.). *Pillow reference*. Pillow Documentation. Recuperado 5 de noviembre de 2026, de https://pillow.readthedocs.io/en/stable/reference/

PyTorch. (s. f.). *ExecuTorch documentation*. PyTorch. Recuperado 5 de noviembre de 2026, de https://pytorch.org/executorch/stable/

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779-788. https://doi.org/10.1109/CVPR.2016.91

Ren, S., He, K., Girshick, R., & Sun, J. (2015). *Faster R-CNN: Towards real-time object detection with region proposal networks* (Preprint 1506.01497). arXiv. https://doi.org/10.48550/arXiv.1506.01497

Rhasspy contributors. (s. f.). *Piper: A fast, local neural text to speech system* [Software]. GitHub. Recuperado 5 de noviembre de 2026, de https://github.com/rhasspy/piper

Roboflow. (s. f.). *Roboflow: Computer vision tools for developers and enterprises*. Roboflow. Recuperado 5 de noviembre de 2026, de https://roboflow.com/

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 4510-4520. https://doi.org/10.1109/CVPR.2018.00474

Szeliski, R. (2011). *Computer vision: Algorithms and applications*. Springer. https://doi.org/10.1007/978-1-84882-935-0

Tan, M., Pang, R., & Le, Q. V. (2020). EfficientDet: Scalable and efficient object detection. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10781-10790. https://doi.org/10.1109/CVPR42600.2020.01079

TensorFlow. (s. f.). *TensorFlow Lite*. TensorFlow. Recuperado 5 de noviembre de 2026, de https://www.tensorflow.org/lite/guide

TensorFlow contributors. (s. f.). *Performance measurement*. GitHub. Recuperado 5 de noviembre de 2026, de https://github.com/tensorflow/tensorflow/blob/master/tensorflow/lite/g3doc/performance/measurement.md

Ultralytics. (s. f.-a). *Explore Ultralytics YOLOv8*. Ultralytics. Recuperado 5 de noviembre de 2026, de https://docs.ultralytics.com/models/yolov8/

Ultralytics. (s. f.-b). *Ultralytics YOLO11*. Ultralytics. Recuperado 5 de noviembre de 2026, de https://docs.ultralytics.com/models/yolo11/

Universidad del Magdalena. (2020). *Plan de Desarrollo UNIMAGDALENA Comprometida 2020–2030*. https://bloque10.unimagdalena.edu.co/wp-content/uploads/2021/10/Plan-de-desarrollo-Unimagdalena-comprometida-2020-2030-1.pdf

Universidad del Magdalena. (2024). *Plan de Gobierno Rectoral 2024-2028*. https://www.unimagdalena.edu.co/Content/Public/Docs/Entrada_10/adjunto_10-20241210150115_739.pdf

World Health Organization. (2019). *World report on vision*.